Unit 1
Creative Development & Collaboration

100-Question Priority-Starred Master Set

- Questions 1–90: Multiple Choice (Full Priority Ratings & Distractor Breakdowns)
- Questions 91–100: Create Performance Task Style Written Responses (9 Points Each)
- Complete Answer Key and Detailed Worked Solutions at the End

Examination Structure

This paper contains 100 continuously numbered questions designed for complete mastery of AP Computer Science Principles Unit 1: Creative Development (Big Idea 1: CRD). All answers, step-by-step worked solutions, and 9-point scoring rubrics appear after Question 100.

★ Priority Rating System

★★★★★ **Very Important**	High-frequency exam concepts: Pair programming roles (Driver vs Navigator), Program Purpose vs Function, Event-Driven Programming, Iterative vs Incremental design, Syntax vs Runtime vs Logic errors, Boundary condition testing, Code attribution & Creative Commons licenses.
★★★★ **High Priority**	Core development mechanics: User input validation, Sensor and audio/haptic outputs, Prototyping workflows, Off-by-one errors, Trace tables, Preconditions & Postconditions, Open-source vs Proprietary licensing, and Semantic versioning.
★★★ **Mandatory**	Essential computing standards: Software development lifecycles, Overflow and round-off errors, Print statement & breakpoint debugging, Code maintainability & self-documenting identifiers, and Branch testing coverage.
★★ **Important**	Foundational applications: Program specification drafting, Peer review etiquette, Version control merge conflicts, and Public domain usage.

Topic Coverage (Big Idea 1: CRD-1 & CRD-2)

1.1 Collaboration (Pair Programming, Team Diversity, Bias Mitigation)

1.2 Program Function & Purpose (User Stories, System Specifications)

1.3 Program Design & Development (Iterative Cycles, Prototyping)

1.4 Event-Driven Architecture (Inputs, Sensors, Event Queues, Outputs)

1.5 Error Typologies (Syntax vs Runtime vs Logic, Overflow, Round-Off)

1.6 Debugging Strategies (Trace Tables, Breakpoints, Edge Cases)

1.7 Code Documentation (Comments, Pre/Postconditions, API Modularity)

1.8 Intellectual Property (Creative Commons, Open Source, Attribution)

> **AP Exam Pro Tip:** Always distinguish between a program's *purpose* (the human or societal problem being solved for users) and its *function* (how the algorithm executes statements, manipulates variables, and produces outputs)! In debugging questions, remember that a **logic error** does NOT cause the compiler to halt or the app to crash; it executes to completion but produces an incorrect result silently!

- **Questions 1–90:** Multiple choice, four options (A–D), exactly one correct answer.
- **Questions 91–100:** Create Performance Task Written Response scenarios, 9 points each, with full rubrics.
- **Difficulty Mix:** Authentic Medium AP Exam Level ❖ Rigorously 3-Pass Verified.
- **Answer Section:** Begins after Question 100 — master answer key, detailed MCQ solutions with distractor failure analysis, and model FRQ solutions with official point allocations.

ITEM 01

TOPIC: PAIR PROGRAMMING ROLES & WORKFLOWS ★★★★★ VERY IMPORTANT

Two software engineers are using the pair programming model to implement a complex sorting routine for a mobile application. Which of the following best describes the specific responsibilities of the 'navigator' in this collaborative process?

- **A** The navigator types the actual lines of code into the IDE, executes test builds, and resolves syntax errors directly.
- **B** The navigator observes the code being written, considers the broader architectural picture, identifies edge-case defects in real time, and suggests strategic improvements.
- **C** The navigator acts exclusively as the final quality assurance tester after the driver has completed the entire program without interruption.
- **D** The navigator is solely responsible for writing end-user documentation and marketing summaries while the driver handles all technical implementation.

ITEM 02

TOPIC: DIVERSITY IN COMPUTING & BIAS MITIGATION ★★★★★ VERY IMPORTANT

A startup company develops a facial recognition application for workplace door security. During initial deployment, the software frequently fails to identify employees with darker skin tones and employees who wear prescription eyeglasses. Which of the following best explains how collaboration could have prevented this issue during early development?

- **A** Including team members and test groups from diverse demographic backgrounds would have ensured the training dataset and algorithmic parameters accounted for varied facial structures, lighting conditions, and skin tones.
- **B** Pair programming between two software engineers with identical technical backgrounds automatically eliminates all algorithmic and dataset biases.
- **C** Assigning the entire computer vision pipeline to a single senior engineer avoids communication overhead and guarantees objective mathematical neutrality.
- **D** Restricting beta testing exclusively to the company's executive committee ensures consistent test data under uniform corporate lighting.

ITEM 03

TOPIC: ASYNCHRONOUS COLLABORATION & VERSION CONTROL ★★★ MANDATORY

A distributed team of high school students is collaborating across different time zones to build an educational web app. Which of the following tools and practices best enables asynchronous collaboration while preventing simultaneous edits from overwriting each other's work?

- **A** Emailing a single compressed .zip file of the project back and forth at the end of each school day.
- **B** Using a centralized or distributed version control system (such as Git) with branch management, commit histories, and pull requests.
- **C** Allowing only one student to touch the codebase while the other teammates dictate code over phone calls.
- **D** Working on independent local copies of files without tracking changes and copying all code into one file at the end of the semester.

ITEM 04

TOPIC: COLLABORATION IN THE SOFTWARE LIFE CYCLE

★★★★★ VERY IMPORTANT

During which phase of the software development life cycle is collaborative user feedback most critical for defining the overarching program purpose and user requirements?

- **A** During the compiling phase when the source code is translated into machine binary instructions.
- **B** During the investigation and reflection phase when developers interview users to understand their challenges, workflows, and goals.
- **C** During the code minification phase right before uploading the app bundle to an application store.
- **D** After the product has reached its end-of-life and software support has been permanently discontinued.

ITEM 05

TOPIC: INTERPERSONAL COMMUNICATION & CONSENSUS

★★★ MANDATORY

Two software developers disagree on the layout of a health tracking dashboard. Developer X wants a dense text table with maximum raw data, while Developer Y advocates for high-level visual charts with drill-down options. According to effective collaborative computing practices, how should the team resolve this conflict?

- **A** The developer with the higher seniority should immediately overrule the other without testing or discussion.
- **B** Develop rapid interactive prototypes of both designs and conduct usability testing with representative end users to gather empirical feedback.
- **C** Abandon the health tracking dashboard entirely and delete the feature from the application roadmap.
- **D** Choose the design that requires the fewest lines of code to implement, regardless of user accessibility.

ITEM 06

TOPIC: CODE REVIEWS & PEER INSPECTION

★★ IMPORTANT

Which of the following represents a primary benefit of conducting collaborative peer code reviews before merging new code into a shared repository?

- **A** Code reviews guarantee that the software will execute with 0% CPU utilization across all client hardware.
- **B** Code reviews allow peers to identify subtle logic errors, enforce consistent coding standards, and share domain knowledge across the team.
- **C** Code reviews eliminate the need for ever writing unit tests or running automated test suites.
- **D** Code reviews permanently prevent any competitor from reverse engineering the deployed binary.

ITEM 07

TOPIC: PAIR PROGRAMMING FAILURE MODES

★★★★★ VERY IMPORTANT

A teacher observes two students attempting pair programming on a simulation project. Student A has been typing code continuously for 45 minutes while Student B is browsing social media on a smartphone and has not spoken. Which principle of pair programming is being violated in this scenario?

A The students are violating the rule that all code must be written in assembly language during pair programming.

B Both partners must actively participate, communicate continuously, and periodically switch between the driver and navigator roles.

C Pair programming requires at least four simultaneous programmers on a single machine to maintain focus.

D Students in pair programming are forbidden from compiling or executing their code until the end of the school term.

ITEM 08

TOPIC: ACCESSIBILITY & INCLUSIVE DESIGN COLLABORATION

★★★ MANDATORY

A civic technology group is designing an online voting information portal for municipal elections. To ensure the application is accessible to citizens with visual impairments, who should the development team collaborate with during the design and evaluation phase?

A Only the database administrators who configure SQL table schemas.

B Accessibility specialists, screen-reader users, and individuals with varied degrees of visual impairment.

C Graphic designers who exclusively specialize in low-contrast neon color schemes.

D Automated server load testers that evaluate HTTP request throughput under high network traffic.

ITEM 09

TOPIC: TASK DECOMPOSITION & DELEGATION

★★ IMPORTANT

When a team of four programmers collaborates on a large software project, which of the following project management strategies best minimizes duplicated effort and prevents merge conflicts?

A Assign all four programmers to write the exact same authentication module simultaneously without communication.

B Decompose the overall application into distinct, modular components with clearly defined input/output interfaces and assign specific modules to team members.

C Require every programmer to write code on the master branch without creating separate development branches.

D Prohibit team members from documenting their functions so that only the original author can run the code.

ITEM 10

TOPIC: COLLABORATIVE DEBUGGING STRATEGIES ★★★★★ VERY IMPORTANT

A programmer has spent three hours trying to locate a subtle off-by-one bug in a recursive function and feels stuck. They invite a teammate to conduct 'rubber duck debugging' or a walkthrough session. Why is explaining the code aloud to a peer an effective collaborative debugging strategy?

A Explaining the code forces the programmer to articulate their assumptions step-by-step, which often exposes flawed logic that was previously overlooked.

B Speaking out loud transmits electrical signals to the computer compiler that automatically repairs pointer arithmetic.

C Teammates can immediately delete the recursive function and replace it with an infinite while loop without analyzing the bug.

D Explaining code aloud is only beneficial if the listener holds a formal doctorate in computer science.

ITEM 11

TOPIC: PROGRAM PURPOSE VS. PROGRAM FUNCTION ★★★★★ VERY IMPORTANT

A student develops a mobile application named 'EcoTracker' that allows users to log their daily public transit usage and calculates their estimated carbon emission savings. Which of the following best distinguishes the program's *purpose* from its *function*?

A The purpose is to encourage environmental sustainability by helping users track carbon savings; the function is how the code accepts transit miles, multiplies by emission constants, and displays the saved kilograms on screen.

B The purpose is to execute arithmetic multiplication in the CPU; the function is to save the planet from climate change.

C The purpose and function are identical concepts in computer science and refer strictly to the compiled binary instructions executed by the operating system.

D The purpose describes the programming language syntax used; the function describes the visual color palette chosen by the graphic designer.

ITEM 12

TOPIC: IDENTIFYING PROGRAM PURPOSE FROM CONTEXT ★★★★★ VERY IMPORTANT

Consider the following description of a software system: 'A community pantry deploys a web tool where neighborhood families can anonymously register their household food allergies, view available surplus pantry items in real time, and reserve customized dietary boxes for weekly pickup.' What is the primary purpose of this program?

A To maximize HTTP network traffic on the local municipal server infrastructure.

B To connect food-insecure families with safe, allergen-compatible groceries efficiently and with dignity.

C To test whether SQL relational databases can store boolean flag values for peanut allergies.

D To enforce strict password complexity rules for volunteer pantry managers.

ITEM 13

TOPIC: PROGRAM FUNCTION BEHAVIOR ★★★ MANDATORY

An e-commerce checkout script contains the following pseudocode segment:

```
IF (cartTotal >= 50.00)
{
    shippingFee <- 0.00
}
ELSE
{
    shippingFee <- 5.99
}
finalCharge <- cartTotal + shippingFee
```

Which of the following describes the *function* of this code segment?

A To eliminate world poverty through digital commerce innovation.

B To evaluate whether cartTotal meets a $50.00 threshold, set shippingFee to either $0.00 or $5.99 accordingly, and compute finalCharge as their sum.

C To encourage consumers to purchase sustainable recycled packaging materials.

D To demonstrate that computers are faster than human cashiers at mental math.

ITEM 14

TOPIC: IMPACT OF USER FEEDBACK ON PROGRAM PURPOSE ★★★ MANDATORY

A developer builds a digital metronome app intended solely for classical piano students. After release, reviews reveal that physical therapists are using the app to help stroke recovery patients practice rhythmic walking exercises, but they struggle with the tiny visual beat indicators. How should the developer respond to this unexpected user feedback?

A Block all physical therapists from downloading the app because medical therapy was not in the original specification.

B Broaden the program purpose to include rhythmic physical therapy and iteratively enhance the function by adding large, high-visibility visual pulses and haptic vibrations.

C Delete the metronome audio engine entirely and replace the app with a static medical dictionary.

D Increase the price of the app ten-fold to discourage non-musicians from using it.

ITEM 15

TOPIC: IDENTIFYING DIVERGENCE BETWEEN PURPOSE & FUNCTION ★★★★★ VERY IMPORTANT

A student creates a program intended to help high schoolers calculate their grade point average (GPA). However, when a student inputs an 'A' grade (worth 4.0 points), the program inadvertently subtracts 4.0 points from the running total instead of adding it. Which of the following statements is correct regarding the program's purpose and function?

A The program's purpose is broken, but its function is executing perfectly.

B The program's purpose remains helping students calculate their GPA, but its actual function contains a logic error that prevents it from fulfilling that purpose.

C The program no longer has a purpose or a function once a runtime bug occurs.

D The compiler automatically alters the program's purpose to become a grade penalty calculator.

ITEM 16

TOPIC: CREATIVE EXPRESSION IN COMPUTING

★★ IMPORTANT

Which of the following projects best exemplifies computing used primarily for *creative expression* rather than purely analytical data processing?

A A banking server script that computes compound daily interest on 100,000 checking accounts.

B An interactive generative art installation that renders evolving fractal patterns and harmonious synthesized tones in response to ambient gallery sounds.

C A script that parses comma-separated employee payroll records and outputs W-2 tax forms.

D A network utility that pings IP addresses to measure round-trip packet latency.

ITEM 17

TOPIC: USER-CENTERED DESIGN PRINCIPLES

★★★ MANDATORY

A team is developing a mobile navigation app specifically tailored for senior citizens. During initial testing, senior users report difficulty reading street names and struggle with complex multi-finger pinch gestures. Which user-centered design modification directly addresses these findings?

A Increasing typography contrast and font scale while replacing multi-finger gestures with prominent, single-tap zoom buttons.

B Converting the entire user interface to command-line text prompts to eliminate visual clutter.

C Removing all map views and requiring users to memorize turn-by-turn coordinate vectors.

D Shortening the battery life of the phone whenever the application runs in background mode.

ITEM 18

TOPIC: SPECIFYING REQUIREMENTS & CONSTRAINTS

★★★★★ VERY IMPORTANT

Before writing any code for a hospital patient monitoring system, the engineering team documents that the software must alert staff within 500 milliseconds of detecting an abnormal heart rhythm and must comply with health privacy regulations (HIPAA). These documented requirements represent:

A System specifications and operational constraints that guide program design and validation.

B Unnecessary bureaucratic obstacles that should be ignored to speed up coding.

C Syntax errors that will prevent the program from compiling.

D Compiler directives that instruct the CPU how to allocate cache memory.

ITEM 19

TOPIC: SOCIAL IMPACT AS PROGRAM PURPOSE

★★★ MANDATORY

Which of the following programs is designed with a primary purpose of addressing the 'digital divide' in education?

A A proprietary virtual-reality chemistry simulator that requires a $3,000 specialized gaming PC and high-speed fiber internet.

B An offline-first educational quiz platform that runs on low-cost refurbished tablets and syncs student progress via local USB drives when internet access is unavailable.

C A subscription-only algorithmic trading bot that trades cryptocurrency options on global financial exchanges.

D A high-frequency video streaming app that charges users per gigabyte of bandwidth consumed.

ITEM 20

TOPIC: PROGRAM FUNCTION VALIDATION

★★ IMPORTANT

A programmer writes a function designed to return the maximum value from a list of positive integers. To verify that the program's *function* accurately satisfies this purpose, which testing strategy should the programmer employ?

A Test the function with varied inputs, including a list with one item, a list where the maximum is first, a list where the maximum is last, and a list with duplicate maximum values.

B Run the function once with the input [5] and assume all possible lists of any size will work correctly.

C Inspect the color theme of the code editor to confirm that variable names are highlighted in purple.

D Change the file extension from .py to .docx to check if word processing software can run the algorithm.

ITEM 21

TOPIC: PROGRAM INPUTS & SENSORS

★★★★★ VERY IMPORTANT

A modern smartphone fitness application tracks a runner's distance, cadence, and heart rate during a workout. Which of the following is considered a physical *sensor input* to this program?

A The total kilometers displayed on the high-resolution OLED screen.

B The accelerometer and GPS hardware chip transmitting raw coordinate and motion data to the operating system.

C The chime sound played through Bluetooth earbuds when completing a mile.

D The summary PDF emailed to the user at the end of the week.

ITEM 22

TOPIC: EVENT-DRIVEN PROGRAMMING CONCEPTS ★★★★★ VERY IMPORTANT

In an event-driven graphical user interface (GUI), what determines the exact order in which different blocks of code execute?

A Code blocks always execute in strict top-to-bottom sequential line order from line 1 to the end of the file, regardless of user interaction.

B Code blocks (event handlers) execute asynchronously in response to triggered events, such as user mouse clicks, keystrokes, timers, or incoming network messages.

C The computer compiler rearranges all functions in alphabetical order by procedure name and runs them once at boot.

D The physical temperature of the CPU chip dictates which function executes next.

ITEM 23

TOPIC: IDENTIFYING PROGRAM OUTPUTS ★★★ MANDATORY

An autonomous greenhouse management system monitors soil moisture and ambient sunlight. When soil moisture drops below 25%, the system activates a water pump valve for 10 seconds and logs the watering event with a timestamp to an administrative text file. In this system, which of the following is an example of an *actuator/physical output*?

A The analog soil moisture sensor probe measuring electrical conductivity in the soil.

B The electrical signal triggering the mechanical water pump valve to open and release water.

C The sunlight entering the greenhouse through the glass roof.

D The 25% threshold constant hardcoded in the source code configuration.

ITEM 24

TOPIC: EVENT HANDLER PSEUDOCODE ANALYSIS ★★★ MANDATORY

Consider the following mobile app event handler:

```
WHEN clickSubmitButton IS CLICKED:
    userName <- getText(nameInputBox)
    IF (userName = "")
    {
        setText(errorLabel, "Name cannot be blank!")
        playSound("buzz.mp3")
    }
    ELSE
    {
        setText(welcomeLabel, "Welcome, " + userName)
        navigateToScreen("HomeScreen")
    }
```

Under what condition will the program produce an auditory sound output?

A Whenever the submit button is clicked and the user has entered at least one character in nameInputBox.

B Whenever the submit button is clicked while nameInputBox contains an empty string ("").

C Continuously every second in an infinite background loop whether the user clicks or not.

D Only when the user navigates successfully to HomeScreen.

ITEM 25

TOPIC: TACTILE/HAPTIC OUTPUTS & ACCESSIBILITY ★★★★★ VERY IMPORTANT

A mobile banking app incorporates a brief vibration (haptic feedback) whenever a user successfully transfers money between accounts, in addition to displaying a green checkmark on the screen. Why is providing multiple modalities of output (visual and tactile) advantageous?

A It allows users who are visually impaired or in bright direct sunlight to receive physical confirmation that their transaction succeeded.

B Haptic vibration increases the transfer limit of the user's bank account by 50%.

C Smartphones cannot process visual graphics without simultaneous vibration.

D It converts the banking application into a video game classified under entertainment regulations.

ITEM 26

TOPIC: SEQUENTIAL VS. EVENT-DRIVEN EXECUTION ★★★ MANDATORY

Which of the following programming scenarios is best implemented using an *event-driven* paradigm rather than a traditional *sequential* script?

A A batch script that reads 5,000 sensor log files from yesterday, computes the average temperature, and saves the result to a database before shutting down.

B An interactive touchscreen kiosk at an airport where travelers tap buttons to look up flight gates, search food options, or call for wheelchair assistance.

C A command-line mathematical utility that computes the factorial of 10 and prints the output directly.

D A script that converts an existing MP4 video file into an animated GIF without user intervention.

ITEM 27

TOPIC: INPUT VALIDATION AS A DEFENSE ★★ IMPORTANT

```
A web registration form asks users for their age. A user types 'twenty-five' instead of the digits '25'. If the program
```

A A runtime error (exception) caused by an invalid string-to-integer conversion format.

B A syntax error that prevents the developer from saving the JavaScript or Python source file.

C An overflow error where the number exceeds 64 bits of memory.

D A physical hardware burnout of the user's computer keyboard.

ITEM 28

TOPIC: EVENT-DRIVEN SHARED STATE CONFLICTS

★★★★★ VERY IMPORTANT

In an online multiplayer quiz game, two players click their respective 'Buzz In' buttons at almost the exact same microsecond. What software mechanism ensures that the program fairly determines which player buzzed in first?

A The program generates a random number between 1 and 10 and assigns the point to whoever has the longest username.

B The event queue (or event loop) processes incoming events sequentially in strict arrival order, timestamping each event to resolve concurrency.

C The program crashes immediately whenever two events happen within the same second.

D The computer operating system shuts down the display to prevent cheating.

ITEM 29

TOPIC: MULTI-SOURCE SENSOR INPUTS

★★★ MANDATORY

An autonomous drone navigation program requires continuous inputs from a barometer (for altitude), a magnetometer (for heading), a GPS receiver (for latitude/longitude), and computer vision cameras (for obstacle detection). Combining multiple sensor inputs to produce a single reliable navigational decision is known as:

A Sensor fusion.

B Binary fragmentation.

C Lossy compression.

D Phishing.

ITEM 30

TOPIC: OUTPUT PERSISTENCE & FILE STORAGE

★★ IMPORTANT

A weather logging program collects temperature readings every 10 minutes. When the program terminates, the developer wants the collected readings to remain available for analysis tomorrow. Where must the program write its output to ensure data persistence after power-down?

A In temporary RAM cache memory variables.

B To non-volatile secondary storage (such as a hard drive, solid-state drive, or cloud database file).

C Inside the CPU registers.

D Onto the monitor's display refresh buffer.

ITEM 31

TOPIC: ITERATIVE VS. INCREMENTAL DEVELOPMENT

★★★★★ VERY IMPORTANT

A software engineering team is building a complex student grading portal. Instead of attempting to code the entire system in one massive pass, they first implement student login and test it thoroughly. Next, they implement grade entry and test it. Then, they add report card generation and test it. Which of the following software development methodologies does this process best exemplify?

A Incremental development, because the program is broken into smaller manageable parts, each of which is built, tested, and added to the existing system sequentially.

B The waterfall model, because all coding, testing, and deployment occurred strictly in a single non-repeating monolithic phase.

C Unplanned development, because writing code in pieces proves the team lacked a coherent final vision.

D Binary refactoring, because the developers only used boolean logic to design the system architecture.

ITEM 32

TOPIC: THE ROLE OF PROTOTYPING

★★★★★ VERY IMPORTANT

Before writing any production backend code for a ride-sharing mobile app, a team creates an interactive clickable wireframe using simple visual mockup tools. They ask prospective riders to navigate the prototype to request a ride. What is the primary purpose of creating a prototype in this phase?

A To test whether the GPS hardware chips inside smartphones can survive high vehicle speeds.

B To explore user interface ideas, uncover usability flaws, and validate user workflows rapidly without investing heavy engineering resources into complex code.

C To completely replace the need for writing server-side algorithms or databases.

D To obtain official copyright protection from the government for the user interface.

ITEM 33

TOPIC: ITERATIVE REFINEMENT CYCLES

★★★ MANDATORY

Which of the following sequences best illustrates a complete *iterative* development cycle for an algorithmic feature?

A Write 10,000 lines of code -> Deploy to app store -> Never read user reviews -> Delete repository.

B Investigate user needs -> Design algorithm -> Prototype implementation -> Test with users and collect feedback -> Refine design based on results.

C Compile code -> Run minifier -> Package APK -> Rename variables.

D Print source code on paper -> Shred the paper -> Rewrite identical code from memory.

ITEM 34

TOPIC: INVESTIGATING & REFLECTING PHASE ★★★ MANDATORY

A student is planning an educational game for elementary school students to learn fractions. During the 'investigation and reflection' stage of development, which of the following activities should the student conduct?

A Interview elementary school math teachers and observe young students solving fraction problems to identify common misconceptions and learning roadblocks.

B Register a limited liability corporation (LLC) in another state and hire corporate tax attorneys.

C Purchase a high-end web server rack and install industrial liquid cooling systems.

D Encrypt their hard drive with military-grade 4096-bit RSA keys.

ITEM 35

TOPIC: DESIGN TOOLS: STORYBOARDS & FLOWCHARTS ★★ IMPORTANT

A developer uses a visual flowchart with diamonds representing conditional IF decisions and rectangles representing sequential processing steps. Why are flowcharts and pseudocode valuable during the *design* phase of software development?

A They allow developers to plan and verify algorithmic logic conceptually without getting bogged down in language-specific syntax rules.

B Flowcharts can be inserted directly into the computer's CPU socket to execute twice as fast as machine code.

C Operating systems require a printed flowchart to be scanned by a webcam before launching any executable program.

D Drawing flowcharts automatically increases the available bandwidth of the local Wi-Fi router.

ITEM 36

TOPIC: BENEFITS OF INCREMENTAL TESTING ★★★★★ VERY IMPORTANT

A programmer adopts a policy of testing every new function immediately after writing it, rather than waiting until all 20 functions in the program are completed. What is the greatest advantage of this incremental testing strategy?

A When a bug occurs, the programmer knows the defect is almost certainly located in the most recently added function, making debugging drastically faster and easier.

B Incremental testing completely eliminates the need for ever running the program on real user devices.

C Testing each function individually causes the computer hard drive to spin backwards, reversing software wear.

D It allows the programmer to skip writing return statements inside functions.

ITEM 37

TOPIC: USER STORIES & ACCEPTANCE CRITERIA ★★★ MANDATORY

In modern software planning, teams frequently write user stories in the format: 'As a [type of user], I want to [perform a task] so that [I achieve a benefit].' Which of the following is the best example of a well-formed user story for a public library app?

A 'As a library member, I want to scan a book's barcode with my camera so that I can instantly check if the book is available in our catalog.'

B 'As a computer chip, I want to receive 5 volts of direct current so that my transistors can toggle state.'

C 'As a software developer, I want to write 10,000 lines of spaghetti code with no comments so that nobody can modify my code.'

D 'As an operating system, I want to throw an unhandled null pointer exception so that the user's phone restarts.'

ITEM 38

TOPIC: REFACTORING IN ITERATIVE DEVELOPMENT ★★ IMPORTANT

After completing a working prototype of a weather app, a developer notices that three different procedures contain nearly identical blocks of code for converting Celsius to Fahrenheit. The developer extracts this repeated code into a single reusable helper function `celsiusToFahrenheit(cTemp)`. This process is known as:

A Code refactoring.

B Phishing.

C Denial of Service.

D Overclocking.

ITEM 39

TOPIC: AGILE VS. WATERFALL IN CREATIVE DEVELOPMENT ★★★★★ VERY IMPORTANT

Why do most modern app development teams favor iterative and agile development frameworks over rigid, upfront waterfall models when building user-facing apps?

A Because user needs, market conditions, and technical discoveries evolve rapidly, and iterative frameworks allow teams to pivot and adapt features based on ongoing feedback.

B Because agile frameworks legally exempt developers from testing their code before deployment.

C Because waterfall models require all software engineers to write code exclusively using binary punch cards.

D Because agile development guarantees that software can be built with zero financial budget and zero electricity.

ITEM 40

TOPIC: DOCUMENTING DESIGN DECISIONS ★★★ MANDATORY

During the development of a collaborative document editor, the team decides to use WebSockets for real-time synchronization instead of standard HTTP polling. They record this decision, along with the reasons and trade-offs considered, in an 'Architecture Decision Record' (ADR). What is the primary benefit of documenting this design decision?

A It ensures that current and future team members understand *why* the technical choice was made, preventing future developers from accidentally undoing the architectural foundation.

B It allows the team to patent the English alphabet.

C It automatically doubles the network speed of the customer's home internet connection.

D It prevents web browsers from needing to render CSS style sheets.

ITEM 41

TOPIC: SYNTAX VS. RUNTIME VS. LOGIC ERROR DEFINITIONS ★★★★★ VERY IMPORTANT

A programmer writes a program to calculate the area of a rectangle. The source code contains the line:

```
area <- length + width
```

The code compiles and runs to completion without crashing, but when length is 5 and width is 4, it outputs 9 instead of the expected 20. What type of error is present in this program?

A A syntax error, because the programming language does not support addition.

B A runtime error, because the operating system aborted the program immediately.

C A logic error, because the program executes completely without crashing but produces an incorrect result due to an algorithmic mistake (using addition instead of multiplication).

D An overflow error, because the number 20 is too large to fit in modern computer memory.

ITEM 42

TOPIC: IDENTIFYING SYNTAX ERRORS ★★★★★ VERY IMPORTANT

A student types the following line into a Python or JavaScript IDE:

```
IF (score >= 90
    DISPLAY("Great job!")
```

When the student clicks 'Run', the environment immediately flags line 1 in red and refuses to execute the program, displaying 'Error: unexpected token; expected closing parenthesis'. What category of error does this represent?

A A syntax error, because the code violates the grammatical structural rules of the programming language.

B A logic error, because the student received an 'A' grade instead of a 'B'.

C A runtime exception caused by zero-division in the ALU.

D A network packet drop caused by wireless router interference.

ITEM 43

TOPIC: IDENTIFYING RUNTIME ERRORS (EXCEPTIONS) ★★★★★ VERY IMPORTANT

Consider the following program segment:

```
numbersList <- [10, 20, 30]
index <- 5
DISPLAY(numbersList[index])
```

Assuming the list indices start at 1, what happens when this code executes?

A The program executes smoothly and prints 50.

B A runtime error occurs (such as an 'IndexOutOfBoundsException' or 'IndexError') because the program attempts to access an element at an index that does not exist in the list.

C A syntax error prevents the file from being saved to the hard drive.

D The computer automatically expands the list to include 50 empty elements.

ITEM 44

TOPIC: DIVIDE-BY-ZERO RUNTIME ERROR ★★★ MANDATORY

A student writes a program to compute the average test score for a classroom. The program contains the statement:

```
average <- totalPoints / studentCount
```

If the program runs on a day when `studentCount` is 0 (e.g., during summer vacation when no students are enrolled), what will occur during execution?

A A runtime error will occur because division by zero is mathematically undefined and cannot be executed by the computer processor.

B The program will automatically set average to infinity without throwing an error.

C A syntax error will prevent the programmer from typing the slash `/` character.

D The computer will format the hard drive to protect system memory.

ITEM 45

TOPIC: OVERFLOW ERRORS ★★★★★ VERY IMPORTANT

```
An embedded system uses a fixed 8-bit unsigned binary integer representation for counting visitors. The maximum integer
```

A A syntax error caused by corrupt ASCII characters.

B An overflow error, because the resulting integer value (256) requires 9 bits (100000000), which exceeds the fixed 8-bit memory allocation.

C A lossy compression artifact.

D A logic error in the mechanical door sensor.

ITEM 46

TOPIC: OFF-BY-ONE LOGIC ERROR ★★★ MANDATORY

A programmer wants to display all integers from 1 to 5 inclusive. They write the following loop in AP pseudocode:

```
count <- 1
REPEAT UNTIL (count = 5)
{
    DISPLAY(count)
    count <- count + 1
}
```

What output does the program actually produce, and what type of error is present?

A Outputs: 1, 2, 3, 4, 5 with no error.

B Outputs: 1, 2, 3, 4 (stops before displaying 5) due to an off-by-one logic error.

C Throws a syntax error on line 2.

D Causes an infinite loop that crashes the operating system.

ITEM 47

TOPIC: TYPE MISMATCH LOGIC/RUNTIME ERROR ★★ IMPORTANT

Consider the following code segment:

```
valA <- "10"
valB <- "20"
result <- valA + valB
DISPLAY(result)
```

If the plus operator `+` acts as string concatenation when applied to text strings, what will be displayed, and why might this be a logic error if the programmer intended numerical addition?

A Displays "1020" because the variables contain string text rather than numeric integers, concatenating the characters instead of performing mathematical addition.

B Displays "30" because computers always convert all text into integers automatically.

C Displays an error message stating that numbers cannot be displayed on a screen.

D Displays "0" because string concatenation deletes all numeric characters.

ITEM 48

TOPIC: BOOLEAN OPERATOR LOGIC ERROR ★★★ MANDATORY

A roller coaster ride has safety requirements: a rider must be at least 48 inches tall AND at least 10 years old. A novice programmer writes the eligibility check as:

```
IF (height >= 48 OR age >= 10)
{
    DISPLAY("Eligible to ride")
}
ELSE
{
    DISPLAY("Not eligible")
}
```

Which of the following test cases reveals the logic error in this code?

A A rider who is 52 inches tall and 12 years old.

B A rider who is 36 inches tall and 6 years old.

C A rider who is 40 inches tall and 11 years old.

D A rider who is 55 inches tall and 15 years old.

ITEM 49

TOPIC: FLOATING-POINT ROUND-OFF ERROR

★★★★★ VERY IMPORTANT

In many programming languages, executing `0.1 + 0.2 == 0.3` evaluates to `false` because the computer calculates `0.30000000000000004`. What computational principle causes this behavior?

A A syntax error caused by using decimal points in source code.

B A round-off error caused by the fact that certain decimal fractions (like 0.1) cannot be represented with exact precision in finite binary floating-point representation.

C A virus installed in the computer's motherboard BIOS.

D An intentional design choice by computer scientists to prevent students from finishing homework early.

ITEM 50

TOPIC: COMPILE-TIME VS. RUN-TIME ERROR DETECTION

★★ IMPORTANT

Which of the following errors is guaranteed to be detected *before* a compiled program ever begins execution?

A A missing closing quotation mark on a string literal on line 42.

B An unexpected divide-by-zero that occurs only when a user types the number zero into an input box.

C An out-of-bounds array access that only happens when a user clicks a button ten times.

D A logic error where the sales tax calculation uses 0.05 instead of 0.06.

ITEM 51

TOPIC: TRACING VARIABLE VALUES WITH A TRACE TABLE

★★★★★ VERY IMPORTANT

Consider the following pseudocode segment:

```
x <- 3
y <- 7
x <- x + y
y <- x - y
x <- x - y
```

What are the values of x and y after this sequence of statements executes?

A x = 3, y = 7

B x = 7, y = 3

C x = 10, y = 10

D x = 0, y = 0

ITEM 52

TOPIC: USING PRINT STATEMENTS FOR DEBUGGING

★★★★★ VERY IMPORTANT

A programmer is debugging a 200-line algorithm that processes customer invoices. The final output is incorrect, but the program does not crash. The programmer inserts temporary `DISPLAY("Check 1: total = ", currentTotal)` statements at multiple intermediate points in the code. What is the primary purpose of this debugging technique?

A To inspect the internal state of variables during execution to pinpoint exactly which statement causes the data to deviate from expected values.

B To increase the processing speed of the CPU by displaying text.

C To permanently alter the program's output to include diagnostic labels for end customers.

D To prevent unauthorized hackers from reading the source code.

ITEM 53

TOPIC: BOUNDARY VALUE TESTING

★★★★★ VERY IMPORTANT

A movie theater ticketing program charges $8 for children aged 12 and under, $14 for adults aged 13 to 64, and $10 for seniors aged 65 and older. To thoroughly test the boundary conditions of the age verification logic, which set of ages should the quality assurance tester be sure to include?

A Only 25 and 40.

B 12, 13, 64, and 65.

C 100, 200, and 300.

D Only negative numbers like -5 and -10.

ITEM 54

TOPIC: TRACE TABLE ANALYSIS WITH LOOPS

★★★ MANDATORY

Consider the following algorithm:

```
total <- 0
k <- 1
REPEAT 3 TIMES
{
    total <- total + (k * 2)
    k <- k + 1
}
DISPLAY(total)
```

What value is displayed after execution?

A 6

B 12

C 14

D 18

ITEM 55

TOPIC: USING BREAKPOINTS IN A DEBUGGER

★★★ MANDATORY

Modern Integrated Development Environments (IDEs) provide interactive debugging tools. A developer sets a 'breakpoint' on line 45 of their code. What happens when the program is run in debug mode?

A The computer permanently deletes line 45 from the hard drive.

B Program execution pauses immediately before line 45 executes, allowing the developer to inspect current variable values in memory and step forward one instruction at a time.

C The computer screen turns off until the developer enters an administrative password.

D The program skips lines 1 through 44 and begins execution at line 45.

ITEM 56

TOPIC: TEST CASE COVERAGE & EDGE CASES

★★ IMPORTANT

A programmer creates a function `findShortestWord(wordList)` that returns the word with the fewest characters from a list of strings. Which of the following represents an important *edge case* that should be included in the test suite?

A A list containing 10,000 words that are all 5 letters long.

B An empty list `[]` containing zero words.

C A list containing words written in three different fonts.

D A list printed on a physical piece of paper.

ITEM 57

TOPIC: ISOLATING DEFECTS BY COMMENTING CODE

★★★★★ VERY IMPORTANT

A 500-line simulation program is failing with an obscure memory crash. The developer temporarily comments out lines 200 through 350. When re-run, the crash disappears. What does this debugging technique demonstrate?

A Code isolation: by systematically disabling sections of code, the developer narrows down the search space to confirm that the bug resides within the disabled block.

B That lines 200 through 350 were written in Latin.

C That commenting out code makes the computer immune to hardware power outages.

D That the compiler automatically rewrites the disabled code to make it faster.

ITEM 58

TOPIC: AUTOMATED UNIT TESTING ★★★ MANDATORY

Why do professional software teams write automated unit test suites that run every time new code is committed to a repository?

A To detect 'regression bugs'—situations where newly added features or bug fixes accidentally break existing, previously working features.

B To satisfy legal requirements that every software program must contain at least one million lines of code.

C To ensure that all computers running the software use the exact same brand of monitor.

D To prevent software developers from ever needing to talk to each other.

ITEM 59

TOPIC: TRACE TABLE WITH NESTED CONDITIONS ★★ IMPORTANT

Consider the following pseudocode:

```
a <- 4
b <- 8
IF (a > 2)
{
    IF (b < 5)
    {
        c <- a + b
    }
    ELSE
    {
        c <- b - a
    }
}
ELSE
{
    c <- a * b
}
DISPLAY(c)
```

What value is displayed?

A 4

B 12

C 32

D 0

ITEM 60

TOPIC: SYSTEMATIC VS. TRIAL-AND-ERROR DEBUGGING ★★★★★ VERY IMPORTANT

A student is experiencing an unexpected calculation bug. Rather than analyzing the algorithm or tracing variables, the student randomly changes plus signs to minus signs, inserts random parentheses, and changes variable names until something changes. Why is this 'shotgun' or trial-and-error approach considered a poor debugging practice?

A It wastes time, rarely addresses the root cause of the bug, and often introduces new, hidden defects that make the program even harder to fix.

B It is illegal under federal copyright law to modify code more than three times per day.

C Changing operators automatically converts the programming language into machine binary.

D Random changes guarantee that the computer CPU will overheat.

ITEM 61

TOPIC: PURPOSE OF PROGRAM DOCUMENTATION

★★★★★ VERY IMPORTANT

Which of the following best describes the primary purpose of writing comprehensive program documentation (such as comments, API descriptions, and README files)?

A To increase the speed at which the computer CPU executes compiled instructions.

B To explain the design, purpose, parameters, and behavior of the code to current and future programmers, enhancing readability and maintainability.

C To prevent the compiler from generating syntax errors when translating code.

D To satisfy a computer's physical requirement that every function must be translated into English before running.

ITEM 62

TOPIC: HOW COMPILERS TREAT COMMENTS

★★★★★ VERY IMPORTANT

A programmer adds multi-line comments explaining a complex encryption algorithm inside a source code file. How does the computer's compiler or interpreter treat these comment lines during execution?

A It completely ignores the comments during translation, so they consume zero CPU execution time and do not alter the program's output.

B It reads every word in the comment and executes it as a system command.

C It pauses execution for three seconds for every line of comments found.

D It converts each comment into an audio voiceover played through the computer speakers.

ITEM 63

TOPIC: SELF-DOCUMENTING CODE & IDENTIFIER NAMING

★★★ MANDATORY

Consider two different implementations of a calculation in a point-of-sale system:

```
Implementation 1:
x <- p * 0.08
y <- p + x
```

```
Implementation 2:
salesTax <- itemPrice * 0.08
totalWithTax <- itemPrice + salesTax
```

Why is Implementation 2 strongly preferred in professional software engineering?

A Implementation 2 runs three times faster on modern 64-bit processors.

B Implementation 2 uses self-documenting, meaningful identifier names that make the code self-explanatory, readable, and less prone to misunderstandings during maintenance.

C Implementation 1 is illegal and will be rejected by all programming language compilers.

D Implementation 2 consumes less hard drive storage space than Implementation 1.

ITEM 64

TOPIC: DOCUMENTING PRECONDITIONS & POSTCONDITIONS ★★★ MANDATORY

A programmer writes the following documentation for a custom mathematical function:

```
// PROCEDURE: calculateSquareRoot(n)
// PRECONDITION: n must be a non-negative real number (n >= 0)
// POSTCONDITION: returns a real number r such that r * r equals n
```

What is the purpose of documenting a 'precondition' for a procedure?

A To state the assumptions or input requirements that must be true *before* the function is called for it to execute correctly.

B To list all the programmers who previously worked on the project.

C To instruct the operating system how many times to reboot the computer.

D To describe what the code should do after the year 2038.

ITEM 65

TOPIC: MODULARITY & PROCEDURAL ABSTRACTION ★★ IMPORTANT

A video game developer divides their program into separate, self-contained procedures: `handlePlayerInput()`, `updatePhysics()`, `checkCollisions()`, and `renderGraphics()`. What is the primary benefit of this modular structure?

A It allows individual components to be developed, tested, and debugged independently, reducing overall system complexity.

B It guarantees that the game will run on computers that do not have a graphics card.

C It prevents players from ever losing a game.

D It allows the developer to avoid writing any conditional statements.

ITEM 66

TOPIC: CONSEQUENCES OF OUTDATED DOCUMENTATION ★★★★★ VERY IMPORTANT

A team updates an e-commerce tax calculation function to handle international Value-Added Tax (VAT), but forgets to update the function's header comments, which still state that the function only handles US sales tax. A new developer joins the team and uses the function incorrectly based on the old comments. What does this scenario illustrate?

A Documentation must be updated in tandem with code changes; out-of-date documentation is actively misleading and can cause serious logic errors.

B Comments should never be written because they always cause software to crash.

C New developers should never be allowed to read code documentation.

D All programming languages automatically delete outdated comments using artificial intelligence.

ITEM 67

TOPIC: WHEN TO WRITE DOCUMENTATION ★★★ MANDATORY

In an effective iterative development workflow, when should documentation and code comments be written?

- **A** Throughout the development process—during planning, as code is being written, and whenever code is modified.
- **B** Exclusively on the final day before the product is permanently retired.
- **C** Never, because clean code should always be kept completely secret from other developers.
- **D** Only if the program fails all unit tests.

ITEM 68

TOPIC: AUDIENCE-APPROPRIATE DOCUMENTATION ★★ IMPORTANT

A software company publishes two distinct documentation manuals for their cloud photo storage app: Manual Alpha explains API endpoints, JSON request schemas, and OAuth authentication tokens; Manual Beta explains how to upload photos, create shared family albums, and reset passwords. Who are the intended audiences for Manual Alpha and Manual Beta?

- **A** Manual Alpha is technical documentation for software developers; Manual Beta is user documentation for everyday consumers.
- **B** Manual Alpha is for children; Manual Beta is for rocket scientists.
- **C** Both manuals are written exclusively for hardware circuit board manufacturers.
- **D** Manual Alpha is an operating system kernel; Manual Beta is a graphics driver.

ITEM 69

TOPIC: API DOCUMENTATION & PROGRAM INTERFACES ★★★★★ VERY IMPORTANT

When a developer uses a third-party mapping library (such as Google Maps API or Leaflet), they read the library's API documentation rather than reading all 50,000 lines of the library's internal source code. What computational concept enables this efficiency?

- **A** Abstraction, because the API documentation explains *how to use* the functions (inputs, outputs, and behaviors) without requiring the developer to understand the internal implementation details.
- **B** Lossy video compression, because the source code was permanently deleted to save hard drive space.
- **C** Phishing, because the library tricks the developer into giving away passwords.
- **D** Bit-level overflow in the graphic rendering card.

ITEM 70

TOPIC: CODE REFACTORING TO IMPROVE DOCUMENTATION ★★★ MANDATORY

A developer inspects a legacy function named `calc(a, b, c)` with no comments and cryptic single-letter variable names. The developer renames the function to `calculateMortgagePayment(loanAmount, interestRate, termYears)` and adds a comment explaining the financial formula used. Which of the following software attributes was directly improved?

A Maintainability and code readability.

B The physical speed of light inside the fiber-optic network cables.

C The screen resolution of the user's laptop monitor.

D The battery capacity of the computer.

ITEM 71

TOPIC: ACKNOWLEDGING EXTERNAL CODE AUTHORSHIP ★★★★★ VERY IMPORTANT

A student is developing an interactive web app and finds an open-source algorithm online that calculates sunrise and sunset times based on latitude and longitude coordinates. The student incorporates this algorithm into their project. According to College Board AP CSP guidelines and professional computing ethics, what must the student do?

A Claim that they personally invented the solar physics algorithm to receive a higher grade.

B Acknowledge the original author by including a comment in the program documentation citing the source, URL, author, and license of the imported code.

C Rename all variables in the algorithm to make it impossible for search engines to identify the original source.

D Delete the author's copyright notice and sell the code as an exclusive proprietary patent.

ITEM 72

TOPIC: OPEN SOURCE VS. PROPRIETARY SOFTWARE ★★★★★ VERY IMPORTANT

Which of the following best contrasts *Open Source Software* (OSS) with *Proprietary Software*?

A Open source software makes its underlying source code freely available for anyone to inspect, modify, and distribute, whereas proprietary software keeps its source code closed and restricted by commercial licensing.

B Open source software is never tested, whereas proprietary software is guaranteed to contain zero bugs.

C Open source software only runs on microchips made of wood, while proprietary software runs on silicon.

D Open source software cannot be downloaded from the internet.

ITEM 73

TOPIC: CREATIVE COMMONS LICENSING ATTRIBUTES

★★★ MANDATORY

A student building an educational history game wants to include photos from an online archive. The photos are licensed under Creative Commons 'CC BY-NC' (Attribution-NonCommercial). What permissions and restrictions apply to the student's use of these photos?

A The student may use and modify the photos freely as long as they attribute the original creator and do not use the project for commercial profit.

B The student must pay a $5,000 royalty fee to the photographer before viewing the images.

C The student can sell the photos on physical t-shirts without naming the original creator.

D The student is strictly forbidden from ever downloading the images to a hard drive.

ITEM 74

TOPIC: PUBLIC DOMAIN DEDICATION (CC0)

★★★★★ VERY IMPORTANT

A game developer searches for sound effects and selects audio clips marked as 'Creative Commons Zero' (CC0) or 'Public Domain'. What does this designation mean for the developer?

A The creator has waived all copyright interests, meaning the developer can use, modify, and distribute the audio for both personal and commercial projects without permission or mandatory attribution.

B The audio clips will self-destruct after being played exactly zero times.

C The developer must surrender their own computer to the government after downloading the clips.

D The developer can only play the audio clips between midnight and 1:00 AM.

ITEM 75

TOPIC: CONSEQUENCES OF SOFTWARE PLAGIARISM

★★★ MANDATORY

In an industry hackathon with a prize of $50,000, Team Z downloads a completed closed-source mobile game from GitHub, strips out the original copyright notices, re-skins the app logo, and submits it as their own work. What are the ethical and legal consequences of this action?

A It constitutes copyright infringement and fraud, subjecting the team to disqualification, reputational damage, and potential civil lawsuits from the original copyright holders.

B It is celebrated as an exemplary demonstration of agile sprint velocity.

C The hackathon organizers are legally required to grant Team Z a government software patent.

D Nothing, because all software found on the internet automatically becomes property of whoever downloads it first.

ITEM 76

TOPIC: SOFTWARE PATENTS VS. COPYRIGHTS

★★ IMPORTANT

What is the primary distinction between a *software copyright* and a *software patent*?

A Copyright protects the specific creative expression of source code text, while a patent protects the underlying novel, useful, and non-obvious algorithmic method or system process.

B Copyright only applies to hardware, while patents only apply to video games.

C Copyright lasts for exactly 24 hours, while a patent lasts forever.

D There is no difference between copyright and patent law in computer science.

ITEM 77

TOPIC: CITING CODE ON THE AP CSP CREATE PERFORMANCE TASK

★★★★★ VERY IMPORTANT

A student preparing their AP Computer Science Principles Create Performance Task uses a third-party open-source charting library to render a bar chart. Which of the following accurately describes the College Board requirements regarding this external code?

A The student can use the external library, but in their written response explaining their program's algorithm, they must describe an algorithm that they personally developed (not the library's internal code) and must cite the external library in their documentation.

B The student will receive an automatic zero if they use any external libraries or functions anywhere in their project.

C The student must claim they wrote the entire 100,000-line charting library from scratch over the weekend.

D The student must pay the College Board a licensing fee for every third-party function imported.

ITEM 78

TOPIC: COPYLEFT (GPL) LICENSING RULES

★★★ MANDATORY

A developer builds an image editing tool incorporating a compression algorithm licensed under the GNU General Public License (GPL / Copyleft). The GPL license requires that any derivative work that incorporates this code must also be released under the same open-source GPL terms. What must the developer do when distributing their software?

A They must make the source code of their entire image editing tool publicly available under the GPL license.

B They can conceal the source code and sell the program as encrypted closed-source proprietary software.

C They must delete the compression algorithm and replace it with random numbers.

D They must email the physical computer to the Free Software Foundation.

ITEM 79

TOPIC: FAIR USE DOCTRINE IN COMPUTING

★★ IMPORTANT

A student creates a non-profit educational program that critiques video game interface design by displaying small 2-second thumbnail clips of copyrighted commercial games alongside critical analysis. Under United States copyright law, which legal doctrine protects this limited use of copyrighted material for educational critique?

A The Fair Use doctrine.

B The Digital Millennium Copyright Blockade.

C The Open Hardware Initiative.

D The Blockchain Consensus Protocol.

ITEM 80

TOPIC: ETHICAL SHARING OF STUDENT CODE

★★★ MANDATORY

Two classmates are working on separate, individual programming assignments. Classmate A has finished early, while Classmate B is stuck on line 10. Classmate A wants to help without violating academic integrity. Which of the following represents the most ethical collaborative assistance?

A Classmate A emails their entire finished project to Classmate B so Classmate B can copy-paste the file and change the author name.

B Classmate A talks through the algorithmic concept on a whiteboard, asks probing questions about Classmate B's logic, and guides them to discover the fix without sharing direct code.

C Classmate A charges Classmate B $20 to complete the assignment for them.

D Classmate A deletes Classmate B's repository so they both receive an incomplete.

ITEM 81

TOPIC: FULL LIFECYCLE DEFECT RESOLUTION

★★★★★ VERY IMPORTANT

A school bus tracking application crashes intermittently on cold mornings. The engineering team follows a systematic troubleshooting process:
1. Replicate the crash using simulated sensor inputs.
2. Locate the line of code causing the runtime exception.
3. Formulate a hypothesis for why the defect occurs.
4. Modify the source code to handle negative temperature values.
5. Run the existing test suite and verify the fix.

Which of the following describes the purpose of step 5?

A Regression testing: ensuring the new code modification fixes the cold-weather defect without breaking existing, previously verified bus tracking features.

B Hardware overclocking: increasing the bus engine's horsepower using software.

C Deleting the repository commit history to hide the fact that a bug ever existed.

D Converting the programming language from Python to machine assembly code.

ITEM 82

TOPIC: BRANCH COVERAGE & TEST SUITE COMPLETENESS ★★★★★ VERY IMPORTANT

Consider the following discount decision procedure:

```
IF (isStudent = true)
{
    discount <- 0.15
}
ELSE
{
    IF (age >= 65)
    {
        discount <- 0.10
    }
    ELSE
    {
        discount <- 0.00
    }
}
```

What is the minimum number of test cases required to achieve 100% branch coverage (ensuring every possible decision path is executed at least once)?

A 1 test case.

B 3 test cases.

C 10 test cases.

D 100 test cases.

ITEM 83

TOPIC: MULTI-ERROR DIAGNOSIS IN CODE SNIPPETS ★★★ MANDATORY

A student writes a program intended to sum all elements in a list of numbers:

```
Line 1: numbers <- [4, 8, 12]
Line 2: sum <- 0
Line 3: FOR EACH num IN numbers
Line 4: {
Line 5:     sum <- sum + num
Line 6: }
Line 7: average <- sum / 0
Line 8: DISPLAY(average)
```

What error will halt execution of this program, and on which line?

A A syntax error on Line 1 because brackets are not permitted in pseudocode.

B A runtime error (divide by zero exception) on Line 7.

C A logic error on Line 5 that causes an infinite loop.

D A hardware GPU failure on Line 8.

ITEM 84

TOPIC: MERGE CONFLICTS IN COLLABORATIVE DEVELOPMENT ★★★ MANDATORY

During a collaborative hackathon, Programmer Alpha and Programmer Beta both pull the latest version of `calculator.js`. At the same time, Alpha modifies line 15 to implement addition, while Beta modifies line 15 to implement multiplication. When they both attempt to push their changes to the shared repository, what occurs?

A The version control system detects a merge conflict on line 15 and pauses the merge, requiring the developers to communicate, inspect the differences, and choose or combine the correct code before committing.

B The computer operating system permanently deletes both programmers' hard drives to punish them.

C The repository randomly deletes half of the functions in the file.

D The compiler executes both addition and multiplication simultaneously on the exact same microsecond.

ITEM 85

TOPIC: SEMANTIC VERSIONING & SOFTWARE UPDATES

★★ IMPORTANT

A software development team adopts Semantic Versioning (SemVer: `MAJOR.MINOR.PATCH`, such as `2.4.1`). They release an update that fixes a critical security bug and patches an off-by-one loop error, without adding any new features or changing public API interfaces. How should the version number be updated?

A Increment the PATCH number (e.g., from `2.4.1` to `2.4.2`).

B Increment the MAJOR number (e.g., from `2.4.1` to `3.0.0`).

C Rename the software to Version 0.0.0.

D Delete the version number entirely.

ITEM 86

TOPIC: TRACE TABLE: ALGORITHM WITH LIST SWAPS

★★★★★ VERY IMPORTANT

Consider the following pseudocode operating on a list with 1-based indexing:

```
items <- [20, 50, 10, 40]
temp <- items[1]
items[1] <- items[3]
items[3] <- temp
```

What is the state of `items` after these statements execute?

A [10, 50, 20, 40]

B [20, 50, 10, 40]

C [10, 10, 20, 40]

D [50, 20, 40, 10]

ITEM 87

TOPIC: ACCESSIBILITY AUDITS IN ITERATIVE REVIEW

★★★ MANDATORY

During an accessibility review of an online educational platform, an auditor notes that images illustrating science experiments lack alternative text (`alt` attributes). Why does adding descriptive alternative text improve accessibility?

A Screen readers synthesize the alt text into speech or braille for visually impaired students who cannot see the visual diagram.

B Alt text makes the image file download 50 times faster over cellular networks.

C Alt text encrypts the image so that search engines cannot crawl it.

D Alt text is required by the computer monitor to display RGB colors.

ITEM 88

TOPIC: CHANGELOGS & USER COMMUNICATION

★★ IMPORTANT

When deploying an updated version of a mobile app, why is it considered best practice to publish a clear 'Changelog' or release notes?

A It informs users and administrators about new features, bug fixes, known issues, and potential breaking changes.

B It is an automated requirement enforced by the central bank before processing credit cards.

C Writing a changelog deletes all previous versions of the software from existence.

D Changelogs prevent computer hard drives from accumulating physical dust.

ITEM 89

TOPIC: COMPLEX TRACE TABLE: ACCUMULATOR WITH CONDITIONAL FILTER

★★★★★ VERY IMPORTANT

Consider the following pseudocode algorithm operating on a list of integers:

```
scores <- [12, -4, 0, 8, -2, 10]
positiveSum <- 0
positiveCount <- 0
```

```
FOR EACH s IN scores
{
    IF (s > 0)
    {
        positiveSum <- positiveSum + s
        positiveCount <- positiveCount + 1
    }
}
avgPositive <- positiveSum / positiveCount
DISPLAY(avgPositive)
```

What value is displayed?

A 10

B 4

C 5

D 30

ITEM 90

TOPIC: DEPRECATION WARNINGS IN SOFTWARE EVOLUTION

★★★ MANDATORY

A software team marks a legacy utility procedure as 'deprecated' in their library documentation and console logs. What does 'deprecated' mean in software engineering?

A The procedure is still functional in the current version, but it is considered obsolete, will be removed in a future release, and developers are urged to migrate to a newer alternative.

B The procedure has been classified as illegal contraband by international police.

C The procedure has been permanently converted into a hardware microchip.

D The procedure executes 1,000 times faster than any other function.

Program Purpose, Function, and User Input Processing in a Healthcare App

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A high school student creates a mobile wellness program called 'HydrateHero' to assist student-athletes in tracking their daily hydration during intense sports seasons.

The application interface includes:
• A numeric text input field (`waterAmountInput`)
• A dropdown selector for measurement units (`"oz"` or `"ml"`)
• A 'Log Intake' button (`logButton`)
• A visual circular progress bar and text display (`dailyTotalLabel`)

The underlying algorithm executes the following event handler:

```
WHEN logButton IS CLICKED:
    rawAmount <- getText(waterAmountInput)
    unitType <- getSelected(unitDropdown)
    IF (isNumber(rawAmount) = true AND rawAmount > 0)
    {
        amountOz <- convertToOunces(rawAmount, unitType)
        dailyTotalOz <- dailyTotalOz + amountOz
        updateProgressBar(dailyTotalOz, dailyGoalOz)
        setText(dailyTotalLabel, dailyTotalOz + " oz / " + dailyGoalOz + " oz")
        IF (dailyTotalOz >= dailyGoalOz)
        {
            playCelebrationSound()
        }
    }
    ELSE
    {
        setText(errorLabel, "Please enter a valid positive number!")
    }
```

Prompt Requirements:

(a) [2 Points]: Identify the overall purpose of the 'HydrateHero' program and explain how it differs from the program's function.

(b) [2 Points]: Identify one user input and one program output from the scenario. Describe the data type and format of each.

(c) [3 Points]: Explain in detail how the program function behaves when a user inputs '500' with unit 'ml', including how data moves from input to visual output.

(d) [2 Points]: Describe what would happen if a user inputs '-15'. Explain which branch of execution runs and how this input validation prevents a logic defect.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Iterative Development & User Feedback Integration in an Accessibility Tool

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A student developer creates an audio reader tool for visually impaired peers that converts photographed book pages into synthesized speech.

During the initial prototype phase (Sprint 1), the developer tested the app exclusively on clear printed text under studio lighting. When deployed for real-world beta testing with three visually impaired classmates (Sprint 2), the testers reported three major roadblocks:
1. The app provided no audio cues to indicate whether the camera was pointed at a page or if the page was upside down.
2. The synthesized voice spoke too quickly and could not be paused or rewound.
3. The dark blue button layout was impossible to find by touch alone.

Prompt Requirements:

(a) [3 Points]: Explain how the developer's experience demonstrates the necessity of iterative development and diverse user testing over non-iterative development.

(b) [2 Points]: Identify two concrete software modifications the developer should implement in the next iteration to directly address user feedback.

(c) [2 Points]: Describe the specific steps of the investigation, design, prototyping, and testing cycle the developer should follow to validate these new features.

(d) [2 Points]: Explain how failing to include visually impaired users during the investigation phase created developer blind spots.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Event-Driven Architecture & Concurrency in an Interactive School Voting Kiosk

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A student body election utilizes two standalone touchscreen kiosks connected to a central election server. When a student approaches a kiosk, the system displays candidates, accepts a vote selection, and executes the following event handler:

```
WHEN castVoteButton IS CLICKED:
    studentId <- getText(idInput)
    candidateSelected <- getSelectedCandidate()

    IF (hasAlreadyVoted(studentId) = false)
    {
        recordVote(candidateSelected)
        markVoted(studentId)
        displayConfirmationScreen(studentId)
        printPaperReceipt()
    }
    ELSE
    {
        displayErrorScreen("Error: This student ID has already cast a ballot!")
        playSound("alert.mp3")
    }
```

Prompt Requirements:

(a) [2 Points]: Identify the trigger event, the inputs, and the outputs of the `castVoteButton` event handler.

(b) [2 Points]: Explain how an event-driven model differs from a sequential batch script in managing student voting arrivals.

(c) [3 Points]: Suppose two students with different IDs tap `castVoteButton` at the exact same millisecond on Kiosk A and Kiosk B. Explain how an event queue ensures fair, deterministic processing.

(d) [2 Points]: Identify one edge case in student ID input that could cause a logic or runtime error, and describe an input validation check to prevent it.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Collaborative Pair Programming, Role Alternation & Bias Elimination in Computer Vision

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

Two computer science students, Developer X and Developer Y, collaborate on a machine learning model to classify recyclable waste items (plastic, glass, paper, aluminum) using a webcam.

During development:
• Developer X acts as the driver for 2 hours straight, writing complex convolutional neural network code without discussing logic.
• Developer Y checks their email and occasionally suggests variable names.
• The training dataset consists exclusively of pristine, perfectly clean beverage cans photographed under bright fluorescent lighting against white paper.

Prompt Requirements:

(a) [3 Points]: Identify two distinct violations of effective pair programming protocols in this team's workflow and describe the corrective action for each.

(b) [2 Points]: Explain the concept of algorithmic/dataset bias in this scenario and predict how the model will perform when deployed in a real school cafeteria.

(c) [2 Points]: How should the developers collaborate with a broader group of school community members to gather a more representative training and testing dataset?

(d) [2 Points]: Describe the specific role the navigator should have played in challenging Developer X's dataset assumptions during the design phase.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Comprehensive Error Classification & Debugging in an E-Commerce Cart Engine

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A student developer writes an e-commerce shopping cart calculation script. During quality assurance testing, three different defects are encountered:

```
Defect 1:
`totalCost <- subTotal + taxRate * subTotal`
When run with subTotal = 100.00 and taxRate = 0.08, the program computes totalCost as 108.00. However, when a promotional d
`totalCost <- subTotal - discount + taxRate * subTotal`
Because tax is calculated on the pre-discount subTotal instead of the discounted price, the customer is overcharged by $0.8
```

```
Defect 2:
`Line 34: IF (customerAge >= 18`
The IDE displays: 'SyntaxError: unexpected end of line; expected closing parenthesis'.
```

```
Defect 3:
`inventoryCount <- getItemInventory("item_9999")`
`ratio <- totalOrders / inventoryCount`
When an item has zero units in stock, `inventoryCount` is 0, causing the program to immediately abort with an 'ArithmeticEx
```

Prompt Requirements:

(a) [3 Points]: Classify each of the three defects (Defect 1, Defect 2, Defect 3) as either a syntax error, a runtime error, or a logic error. Justify each classification.

(b) [2 Points]: Explain why Defect 1 is typically the most dangerous and difficult type of error to discover in software development.

(c) [2 Points]: Provide the corrected mathematical pseudocode statement for Defect 1 to ensure tax is applied strictly to the post-discount price.

(d) [2 Points]: Write an input validation/guard statement for Defect 3 that prevents the runtime crash when inventoryCount is zero.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Systematic Testing, Boundary Conditions & Test Suite Coverage in a Ride-Share Fare Calculator

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A student develops a ride-sharing fare calculation algorithm:

```
PROCEDURE calculateFare(distanceMiles, isRushHour)
{
    baseFare <- 3.50
    IF (distanceMiles <= 0)
    {
        RETURN -1
    }
    IF (distanceMiles <= 5)
    {
        mileageFee <- distanceMiles * 2.00
    }
    ELSE
    {
        mileageFee <- (5 * 2.00) + ((distanceMiles - 5) * 1.50)
    }
    totalFare <- baseFare + mileageFee
    IF (isRushHour = true)
    {
        totalFare <- totalFare * 1.25
    }
    RETURN totalFare
}
```

Prompt Requirements:

(a) [2 Points]: Identify the purpose and return type of the `calculateFare` procedure.

(b) [3 Points]: Design a test suite containing four distinct test cases: one invalid input, one short trip, one long trip during normal hours, and one rush hour trip. State the inputs and expected return value for each.

(c) [2 Points]: Identify the boundary values for `distanceMiles` in this algorithm and evaluate why testing `distanceMiles = 5` and `distanceMiles = 5.1` is critical.

(d) [2 Points]: Calculate the exact fare returned for `calculateFare(10, true)`. Show all intermediate calculations.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Procedural Abstraction & Modular Decomposition in an Environmental Sensor Array

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A team of high school students builds an environmental monitoring buoy deployed in a local lake. The buoy measures water temperature, pH level, dissolved oxygen, and turbidity every 15 minutes.

Originally, the students wrote a single 400-line script inside an infinite loop that read raw sensor pins, calibrated voltage, formatted text, and sent cellular data packets all mixed together in one continuous block.

In Sprint 3, they refactored the program into distinct procedures:
• `readRawSensors()`
• `calibrateReadings(rawVoltages)`
• `detectPollutionThresholds(calibratedData)`
• `transmitTelemetry(payload)`

Prompt Requirements:

(a) [2 Points]: Explain the concept of procedural abstraction and describe how the refactored design demonstrates it.

(b) [3 Points]: Describe two specific advantages this modular decomposition provides when multiple team members collaborate on the buoy software.

(c) [2 Points]: If the pH sensor hardware is replaced with a new model requiring a different calibration formula, identify which procedure must be modified and explain why the other three procedures remain unaffected.

(d) [2 Points]: Explain how procedural abstraction facilitates unit testing before deploying the buoy into the lake.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Trace Table Analysis, State Verification & Defect Isolation in a Grade Filtering Algorithm

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A teacher writes an algorithm to find the highest test score in a class, excluding scores from students who were marked absent (represented by `-1`).

```
PROCEDURE findHighestScore(scoresList)
{
    maxScore <- 0
    FOR EACH s IN scoresList
    {
        IF (s > maxScore)
        {
            maxScore <- s
        }
    }
    RETURN maxScore
}
```

The teacher tests the procedure with three different test lists:
• Test 1: `scoresList = [75, 92, 88, 95, 81]` -> returns 95
• Test 2: `scoresList = [-1, 84, -1, 91, 78]` -> returns 91
• Test 3: `scoresList = [-1, -1, -1]` -> returns 0

Prompt Requirements:

(a) [2 Points]: Analyze Test 3: Explain why returning `0` represents a logic error in the context of student grades where all students were absent.

(b) [3 Points]: Construct a complete step-by-step trace table for Test 2, showing the values of `s`, the boolean evaluation of `(s > maxScore)`, and `maxScore` after each iteration.

(c) [2 Points]: Modify the algorithm so that it returns `"No scores present"` if all values in the list are `-1` or if the list is empty.

(d) [2 Points]: Explain how initializing `maxScore <- 0` would create a defect if the function were repurposed to track sub-zero temperatures (e.g., `[-15, -8, -22]`).

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

Intellectual Property, Open Source Licensing & Code Attribution in a Robotics Controller

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

A robotics student, Carlos, is programming an autonomous rover for a regional science fair. To control the rover's stepper motors, Carlos downloads a public open-source library called `AccurateStepper` from GitHub, which is licensed under the MIT License.

The MIT License states:
'Permission is hereby granted, free of charge, to any person obtaining a copy of this software... to deal in the Software without restriction... subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.'

Carlos copies the 300 lines of motor timing code into his project file, deletes the author's copyright header, changes function names from `stepMotor` to `carlosStep`, and writes in his project report: 'I designed and programmed a novel motor timing algorithm from scratch.'

Prompt Requirements:

(a) [2 Points]: Identify two distinct ethical and legal violations committed by Carlos in this scenario.

(b) [3 Points]: Explain how Carlos should have properly integrated and cited the `AccurateStepper` library while maintaining full academic integrity.

(c) [2 Points]: Contrast the permissive MIT License with a copyleft license like the GNU General Public License (GPL).

(d) [2 Points]: If Carlos were submitting this project for the AP CSP Create Performance Task, explain what portions of code he could and could not submit as his own algorithm.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

End-to-End Creative Development Lifecycle: From Investigation to Maintenance in a Disaster Relief Portal

★★★★★ CED STANDARD

9 Points Total

Program Scenario & System Architecture:

Following a severe hurricane, a community tech alliance collaborates to build an emergency coordination web application called 'ReliefLink' that matches displaced families with volunteers offering temporary shelter, clean drinking water, and medical supplies.

The development team documents their journey across the four College Board Creative Development phases:
1. Investigation & Reflection
2. Design
3. Prototyping & Incremental Coding
4. Testing, Deployment & Ongoing Maintenance

Prompt Requirements:

(a) [2 Points]: Describe the specific activities the team should carry out during Phase 1 (Investigation & Reflection) to ensure ReliefLink meets the genuine urgent needs of storm victims and rescue workers.

(b) [2 Points]: During Phase 2 (Design), explain why developing low-fidelity paper wireframes and data flowcharts before coding prevents critical miscommunications.

(c) [3 Points]: During Phase 3, explain how the team can use incremental development to deploy a minimal working version (MVP) within 24 hours while continuing to add advanced features over the following week.

(d) [2 Points]: During Phase 4, describe two testing strategies the team must use before public release to ensure the platform handles cellular network drops and sudden traffic surges.

Exam Strategy & Reminders: *Be explicit when referencing variable names and UI element identifiers. Distinguish clearly between what the code accomplishes for the user (purpose) versus how the algorithm operates internally (function).*

End of Question Paper

You have reached the end of Questions 1 through 100.

> **Before you continue:** Make sure you have genuinely attempted every question under timed examination conditions. The pages that follow contain the complete multiple choice master answer key, fully worked step-by-step solutions with detailed distractor failure analysis, and model Create Performance Task written responses with official 9-point AP scoring rubrics.

What Comes Next

❖ **Answer Key (Page 44):** Letter answers for Questions 1–90 with priority star ratings.
❖ **Multiple Choice Solutions (Pages 45–74):** Step-by-step reasoning for Questions 1–90 plus an explanation of why each incorrect option fails.
❖ **Free Response Solutions (Pages 75–84):** Complete model answers for Questions 91–100 with point-by-point 9-point rubrics.
❖ **Appendix & Reference Sheet (Page 85):** Essential AP CSP Pseudocode Conventions, Error Typologies & Licensing Frameworks.
❖ **Student Tribute (Page 86):** Premium "Thank You, Future Computer Scientists!" dedication certificate.

Multiple Choice Master Key

Quick reference key for Questions 1 through 90 with priority star ratings. Complete worked solutions begin on the next page.

Q1 B ★★★★★	Q2 A ★★★★★	Q3 B ★★★	Q4 B ★★★★★	Q5 B ★★★	Q6 B ★★	Q7 B ★★★★★	Q8 B ★★★	Q9 B ★★	Q10 A ★★★★★
Q11 A ★★★★★	Q12 B ★★★★★	Q13 B ★★★	Q14 B ★★★	Q15 B ★★★★★	Q16 B ★★	Q17 A ★★★	Q18 A ★★★★★	Q19 B ★★★	Q20 A ★★
Q21 B ★★★★★	Q22 B ★★★★★	Q23 B ★★★	Q24 B ★★★	Q25 A ★★★★★	Q26 B ★★★	Q27 A ★★	Q28 B ★★★★★	Q29 A ★★★	Q30 B ★★
Q31 A ★★★★★	Q32 B ★★★★★	Q33 B ★★★	Q34 A ★★★	Q35 A ★★	Q36 A ★★★★★	Q37 A ★★★	Q38 A ★★	Q39 A ★★★★★	Q40 A ★★★
Q41 C ★★★★★	Q42 A ★★★★★	Q43 B ★★★★★	Q44 A ★★★	Q45 B ★★★★★	Q46 B ★★★	Q47 A ★★	Q48 C ★★★	Q49 B ★★★★★	Q50 A ★★
Q51 B ★★★★★	Q52 A ★★★★★	Q53 B ★★★★★	Q54 B ★★★	Q55 B ★★★	Q56 B ★★	Q57 A ★★★★★	Q58 A ★★★	Q59 A ★★	Q60 A ★★★★★
Q61 B ★★★★★	Q62 A ★★★★★	Q63 B ★★★	Q64 A ★★★	Q65 A ★★	Q66 A ★★★★★	Q67 A ★★★	Q68 A ★★	Q69 A ★★★★★	Q70 A ★★★
Q71 B ★★★★★	Q72 A ★★★★★	Q73 A ★★★	Q74 A ★★★★★	Q75 A ★★★	Q76 A ★★	Q77 A ★★★★★	Q78 A ★★★	Q79 A ★★	Q80 B ★★★
Q81 A ★★★★★	Q82 B ★★★★★	Q83 B ★★★	Q84 A ★★★	Q85 A ★★	Q86 A ★★★★★	Q87 A ★★★	Q88 A ★★	Q89 A ★★★★★	Q90 A ★★★

90	100%	3-Pass	10 FRQs
TOTAL MCQS	CED ALIGNED	VERIFIED	90 PTS TOTAL

Question 1: Pair Programming Roles & Workflows

★★★★★ VERY IMPORTANT

Correct Answer: (B) — The navigator observes the code being written, considers the broader architectural picture, identifies edge-case defects in real time, and suggests strategic improvements.

STEP-BY-STEP EXPLANATION

- In the formal pair programming paradigm, two programmers share a single workstation (or screen).
- The 'driver' physically controls the keyboard and mouse, focusing on the mechanics of writing syntactically correct code.
- The 'navigator' focuses on the strategic direction, reviewing each line of code as it is typed, checking for logic flaws and boundary bugs, and anticipating upcoming architectural challenges.
- Regularly switching roles between the driver and navigator produces higher-quality code with fewer defects than programming in isolation.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Typing code and managing the keyboard is the direct responsibility of the 'driver', not the navigator.

✗ **(C):** Pair programming requires continuous, real-time collaboration; the navigator does not wait until the driver finishes before starting their review.

✗ **(D):** The navigator actively engages with technical design and algorithmic logic rather than limiting themselves to marketing or administrative documentation.

Question 2: Diversity in Computing & Bias Mitigation

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Including team members and test groups from diverse demographic backgrounds would have ensured the training dataset and algorithmic parameters accounted for varied facial structures, lighting conditions, and skin tones.

STEP-BY-STEP EXPLANATION

- Algorithmic bias frequently arises when software development teams lack diversity and subsequently use narrow, non-representative training datasets or assumptions.
- By collaborating with diverse developers and broad demographic test groups during the investigative and design phases, teams identify blind spots and unrepresentative data samples early.
- College Board Big Idea 1 (CRD-1.C) explicitly emphasizes that consultation and collaboration with diverse stakeholders reduce human bias in computing innovations.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Engineers with identical backgrounds often share the same blind spots; pairing does not remove bias unless diverse perspectives and diverse datasets are intentionally incorporated.

✗ **(C):** A single engineer is highly prone to individual unconscious biases and cannot validate broad demographic compatibility alone.

✗ **(D):** Testing only on a small, non-diverse corporate group exacerbates selection bias rather than eliminating real-world deployment errors.

Question 3: Asynchronous Collaboration & Version Control

★★★ MANDATORY

Correct Answer: (B) — Using a centralized or distributed version control system (such as Git) with branch management, commit histories, and pull requests.

STEP-BY-STEP EXPLANATION

- Asynchronous collaboration occurs when team members work at different times and locations without real-time face-to-face interaction.
- Version control systems provide branch isolation, transparent commit logs, and merge conflict resolution, ensuring multiple developers can contribute concurrently without overwriting files.
- Pull requests allow structured peer review and collaborative feedback before code is integrated into the main production branch.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Emailing zip archives lacks conflict detection, commit tracking, and granular version history, inevitably causing overwritten code.

✗ **(C):** Restricting implementation to one person bottlenecking the project is not effective asynchronous collaboration and fails to leverage team capacity.

✗ **(D):** Merging untracked code at the end of a project produces massive integration conflicts and impossible-to-trace logic bugs.

Question 4: Collaboration in the Software Life Cycle

★★★★★ VERY IMPORTANT

Correct Answer: (B) — During the investigation and reflection phase when developers interview users to understand their challenges, workflows, and goals.

STEP-BY-STEP EXPLANATION

- The software development life cycle begins with investigation, reflection, and requirement gathering.
- Collaborating with end users, domain experts, and prospective clients during the investigation phase ensures that the program purpose directly solves genuine user needs.
- Failing to collaborate early often leads to building a technically sound program that ultimately fails because it solves the wrong problem.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Compiling is an automated machine process performed by software compilers, not a phase for user collaboration.
✗ **(C):** Minification is an automated optimization step for reducing file size and bandwidth, requiring no user requirement discovery.
✗ **(D):** Post-discontinuation analysis cannot retroactively define or correct the active purpose and requirements of a live system.

Question 5: Interpersonal Communication & Consensus

★★★ MANDATORY

Correct Answer: (B) — Develop rapid interactive prototypes of both designs and conduct usability testing with representative end users to gather empirical feedback.

STEP-BY-STEP EXPLANATION

- Effective collaboration in software development relies on objective consensus building, data-driven decisions, and user-centric evaluation.
- Creating rapid prototypes allows developers to test competing hypotheses directly with target users and measure usability metrics (e.g., task completion time, clarity).
- Empirical user testing resolves subjective disagreements constructively while aligning the software with genuine user requirements.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Seniority-based overrides stifle innovation, disregard valid usability arguments, and weaken collaborative team dynamics.
✗ **(C):** Deleting core features simply because of a design disagreement harms the final utility of the software.
✗ **(D):** Minimizing lines of code is a secondary engineering preference that must never take precedence over user accessibility and task fulfillment.

Question 6: Code Reviews & Peer Inspection

★★ IMPORTANT

Correct Answer: (B) — Code reviews allow peers to identify subtle logic errors, enforce consistent coding standards, and share domain knowledge across the team.

STEP-BY-STEP EXPLANATION

- Peer code reviews provide a second pair of eyes to spot algorithmic oversights, unhandled boundary conditions, and security vulnerabilities.
- They ensure adherence to institutional style guides, making code readable and maintainable for future contributors.
- They also facilitate mentorship and team knowledge transfer so that no single developer is an irreplaceable single point of failure.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Code reviews evaluate correctness and architecture; they cannot alter fundamental physical computing constraints or produce zero CPU usage.
✗ **(C):** Code reviews supplement automated testing; they never replace unit, integration, and regression test suites.
✗ **(D):** Code reviews occur internally during development and have no impact on compiled binary disassembly or reverse engineering.

Question 7: Pair Programming Failure Modes

★★★★★ VERY IMPORTANT

Correct Answer: (B) — Both partners must actively participate, communicate continuously, and periodically switch between the driver and navigator roles.

STEP-BY-STEP EXPLANATION

- Pair programming succeeds only when both participants actively engage in their respective roles and communicate thoughts aloud.
- The navigator must stay vigilant, reviewing logic, verifying variable names, and thinking ahead.
- Teams should switch roles at structured intervals (e.g., every 15–20 minutes) to keep cognitive fatigue low and engagement high.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Pair programming is language-agnostic and widely practiced in high-level languages like Python, JavaScript, and Java.
✗ **(C):** Pair programming by definition involves exactly two developers (a pair); four programmers constitutes ensemble or mob programming.
✗ **(D):** Frequent incremental testing and running of code is an essential hallmark of agile pair programming.

Question 8: Accessibility & Inclusive Design Collaboration

★★★ MANDATORY

Correct Answer: (B) — Accessibility specialists, screen-reader users, and individuals with varied degrees of visual impairment.

STEP-BY-STEP EXPLANATION

- Inclusive design requires collaborating directly with users who have diverse physical and cognitive capabilities.
- Working with screen-reader users and accessibility experts reveals navigation barriers (e.g., missing alt text, poor keyboard tab indexing, low color contrast) that sighted developers often overlook.
- This collaborative testing ensures compliance with universal design standards like WCAG (Web Content Accessibility Guidelines).

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Database administrators handle server-side storage and have no direct role in front-end assistive technology evaluation.
✗ **(C):** Low-contrast neon schemes actively impair accessibility for visually impaired and color-blind users.
✗ **(D):** Network throughput stress testers measure server scaling, not user interface accessibility.

Question 9: Task Decomposition & Delegation

★★ IMPORTANT

Correct Answer: (B) — Decompose the overall application into distinct, modular components with clearly defined input/output interfaces and assign specific modules to team members.

STEP-BY-STEP EXPLANATION

- Modular decomposition breaks a large, complex system into smaller, self-contained modules or procedures.
- By establishing clear interface contracts (what inputs a module expects and what outputs it returns), developers can work on separate modules in parallel without interfering with one another.
- This separation of concerns reduces merge collisions and allows targeted unit testing.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Assigning identical tasks to multiple programmers without coordination wastes team resources and generates irreconcilable duplicate code.
✗ **(C):** Committing directly to the master branch without feature branches causes constant merge conflicts and destabilizes working code.
✗ **(D):** Lack of documentation prevents collaboration and cripples the team's ability to maintain or integrate code.

Question 10: Collaborative Debugging Strategies

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Explaining the code forces the programmer to articulate their assumptions step-by-step, which often exposes flawed logic that was previously overlooked.

STEP-BY-STEP EXPLANATION

- Explaining code line-by-line to another person (or even an inanimate object like a rubber duck) forces the developer to slow down and verbalize their mental model.
- Discrepancies between what the developer *thinks* the code does and what the code *actually* does become glaringly obvious during this verbal breakdown.
- The collaborator can ask clarifying questions, test edge cases, and challenge unexamined assumptions.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Spoken words have zero physical or compiler-level interaction with software execution.
✗ **(C):** Replacing a buggy function with an uncontrolled infinite loop introduces severe crashes and demonstrates poor engineering.
✗ **(D):** The cognitive benefit of verbalizing code logic does not depend on the listener's academic credentials; simply structuring the explanation reveals the flaw.

Question 11: Program Purpose vs. Program Function

★★★★★ VERY IMPORTANT

Correct Answer: (A) — The purpose is to encourage environmental sustainability by helping users track carbon savings; the function is how the code accepts transit miles, multiplies by emission constants, and displays the saved kilograms on screen.

STEP-BY-STEP EXPLANATION

- College Board Learning Objective CRD-2.A explicitly distinguishes between program purpose and program function.
- The *purpose* of a program explains *why* the software exists—the real-world problem it solves, the user need it meets, or the creative goal it achieves (e.g., promoting sustainable transit choices).
- The *function* of a program describes *what* the software actually does and *how* it behaves during execution (e.g., reading numeric inputs, executing formulas, updating UI labels).
- Confusing purpose with mechanical function is one of the most common student errors on the AP CSP exam.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** This reverses the two concepts: arithmetic operations are internal functional mechanics, while environmental sustainability is the broader societal purpose.
✗ **(C):** Purpose and function are fundamentally distinct; purpose is user- and problem-centric, whereas function is behavioral and operational.
✗ **(D):** Programming syntax and UI color palettes are implementation details, neither of which defines overarching program purpose.

Question 12: Identifying Program Purpose from Context

★★★★★ VERY IMPORTANT

Correct Answer: (B) — To connect food-insecure families with safe, allergen-compatible groceries efficiently and with dignity.

STEP-BY-STEP EXPLANATION

- The primary purpose of a software system addresses the underlying human or societal problem being solved.
- Here, the program facilitates the safe, anonymous, and efficient distribution of allergen-appropriate food to community members in need.
- Technical operations like HTTP transactions or database querying are mere functional tools to accomplish this human-centered purpose.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Generating network traffic is a physical operational byproduct, never the purposeful objective of civic software.
✗ **(C):** Testing database data types describes an engineering technical check, not the reason the pantry deployed the system.
✗ **(D):** Password security is a routine administrative subsystem, not the primary purpose of the food pantry application.

Question 13: Program Function Behavior

★★★ MANDATORY

Correct Answer: (B) — To evaluate whether cartTotal meets a $50.00 threshold, set shippingFee to either $0.00 or $5.99 accordingly, and compute finalCharge as their sum.

STEP-BY-STEP EXPLANATION

- Program function describes the concrete algorithmic behavior of the code when executed with specific inputs.
- This code segment checks a conditional relational expression (cartTotal >= 50.00), branches execution to assign the shipping fee variable, and calculates the total charge via addition.
- This step-by-step operational narrative precisely articulates program function.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** This is an exaggerated, unrelated aspiration that has nothing to do with this code's operational mechanics.
✗ **(C):** Packaging sustainability is not addressed or influenced by this conditional fee assignment algorithm.
✗ **(D):** Comparing computer speed to human cognition is a philosophical observation, not a description of the code's execution behavior.

Question 14: Impact of User Feedback on Program Purpose

★★★ MANDATORY

Correct Answer: (B) — Broaden the program purpose to include rhythmic physical therapy and iteratively enhance the function by adding large, high-visibility visual pulses and haptic vibrations.

STEP-BY-STEP EXPLANATION

- In modern software development, unexpected user adoption often reveals valuable new program purposes.
- Developers should embrace real-world user feedback by refining the program's scope and iteratively adapting its functionality (e.g., adding accessibility features, larger visual cues, haptic feedback) to meet diverse user needs.
- This iterative responsiveness is a core competency in College Board Big Idea 1 (Creative Development).

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Artificially restricting eager users undermines software adoption and ignores a high-impact creative opportunity.
✗ **(C):** Deleting working audio functionality destroys the core product rather than improving it.
✗ **(D):** Punitive pricing drives away users instead of solving accessibility and usability challenges.

Question 15: Identifying Divergence Between Purpose & Function

★★★★★ VERY IMPORTANT

Correct Answer: (B) — The program's purpose remains helping students calculate their GPA, but its actual function contains a logic error that prevents it from fulfilling that purpose.

STEP-BY-STEP EXPLANATION

- The *purpose* is the intended goal defined by the developer (calculating GPA for students). A bug in the code does not change what the program was intended to do.
- The *function* is what the code actually does upon execution (in this case, executing subtraction instead of addition due to a logic error).
- Because the functional behavior deviates from the intended purpose, the developer must debug the arithmetic logic.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The purpose is what the creator intended (which remains valid); it is the function that is malfunctioning.
✗ **(C):** Programs always have functional behavior (even if flawed) and retain their designer's intended purpose.
✗ **(D):** Compilers strictly translate code into machine instructions; they possess no human agency to modify program purpose.

Question 16: Creative Expression in Computing

★★ IMPORTANT

Correct Answer: (B) — An interactive generative art installation that renders evolving fractal patterns and harmonious synthesized tones in response to ambient gallery sounds.

STEP-BY-STEP EXPLANATION

- Computing is not limited to business automation and mathematics; it is a powerful medium for personal, artistic, and creative expression (CRD-1.A).
- Interactive generative art combining visual fractals, audio synthesis, and microphone responsiveness represents creative computing.
- Payroll scripts, interest calculators, and network latency testers are traditional analytical data processing utilities.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Interest computation is a standard financial mathematical calculation.
✗ **(C):** Tax form generation is a routine administrative compliance process.
✗ **(D):** Network ping testing is a diagnostic systems administration task.

Question 17: User-Centered Design Principles

★★★ MANDATORY

Correct Answer: (A) — Increasing typography contrast and font scale while replacing multi-finger gestures with prominent, single-tap zoom buttons.

STEP-BY-STEP EXPLANATION

- User-centered design tailors the software's functional interface to the specific physical, perceptual, and cognitive needs of its target audience.
- For seniors who may experience reduced visual acuity or dexterity challenges, large high-contrast fonts and simple single-tap target buttons drastically improve usability and reduce frustration.
- Listening to user feedback during prototype testing is essential for creating accessible, effective computing innovations.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Command-line interfaces require technical syntax knowledge and would severely degrade usability for everyday users.
✗ **(C):** Requiring users to memorize coordinates creates an impossible cognitive burden, destroying the app's usefulness.
✗ **(D):** Depleting battery life is a severe performance defect, not a user-centered design improvement.

Question 18: Specifying Requirements & Constraints

★★★★★ VERY IMPORTANT

Correct Answer: (A) — System specifications and operational constraints that guide program design and validation.

STEP-BY-STEP EXPLANATION

- In professional software engineering, specifications describe *what* the system must accomplish (e.g., latency under 500 ms) and constraints describe the boundaries within which it must operate (e.g., legal privacy standards, memory limits).
- Documenting specifications and constraints during the investigation and design phases ensures the software meets safety, regulatory, and performance thresholds.
- Testing against these specifications validates whether the final program function fulfills its critical purpose.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Ignoring specifications in critical domains like healthcare leads to fatal software failures and legal liabilities.
✗ **(C):** Specifications are written documentation in natural language, not syntax bugs in source code.
✗ **(D):** Compiler directives are specific programming language keywords (like #pragma or inline), not high-level system requirements.

Question 19: Social Impact as Program Purpose

★★★ MANDATORY

Correct Answer: (B) — An offline-first educational quiz platform that runs on low-cost refurbished tablets and syncs student progress via local USB drives when internet access is unavailable.

STEP-BY-STEP EXPLANATION

- The 'digital divide' refers to the socioeconomic gap between those who have ready access to computers and high-speed internet and those who do not.
- A lightweight platform that functions offline on inexpensive hardware directly mitigates the digital divide by bringing educational resources to underserved or rural students without reliable broadband.
- High-cost VR setups and paid streaming apps exacerbate the digital divide rather than narrowing it.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Expensive hardware and high-bandwidth requirements exclude low-income students, widening the digital divide.
✗ **(C):** Cryptocurrency trading bots serve financial markets and have no educational equity purpose.
✗ **(D):** Usage-based fee models penalize students with limited financial resources.

Question 20: Program Function Validation

★★ IMPORTANT

Correct Answer: (A) — Test the function with varied inputs, including a list with one item, a list where the maximum is first, a list where the maximum is last, and a list with duplicate maximum values.

STEP-BY-STEP EXPLANATION

- Verifying program function requires structured testing across varied scenarios, edge cases, and boundary conditions.
- Testing with a single-element list, lists with maximums at the front, middle, and end, and duplicate values confirms that the algorithm behaves correctly regardless of data ordering.
- A single trivial test case (like [5]) cannot expose logic flaws such as off-by-one errors or indexing bugs.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Testing only one trivial case is insufficient to validate algorithmic correctness across general inputs.
✗ **(C):** Editor syntax highlighting colors have no connection to the mathematical correctness of algorithm execution.
✗ **(D):** Word processors do not execute source code; changing extensions corrupts file associations.

Question 21: Program Inputs & Sensors

★★★★★ VERY IMPORTANT

Correct Answer: (B) — The accelerometer and GPS hardware chip transmitting raw coordinate and motion data to the operating system.

STEP-BY-STEP EXPLANATION

- Program inputs are data received by a program from users, files, network streams, or physical sensors.
- Accelerometers, GPS receivers, gyroscopes, and optical heart rate monitors are hardware sensors that generate continuous input data streams for software applications.
- Visual screens, audio chimes, and generated PDF reports are outputs, not inputs.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Information displayed on a screen is a visual output, not an input.
✗ **(C):** Chime sounds played through earbuds are auditory outputs.
✗ **(D):** Generated emails and PDF reports are persisted digital outputs.

Question 22: Event-Driven Programming Concepts

★★★★★ VERY IMPORTANT

Correct Answer: (B) — Code blocks (event handlers) execute asynchronously in response to triggered events, such as user mouse clicks, keystrokes, timers, or incoming network messages.

STEP-BY-STEP EXPLANATION

- In event-driven programming (CRD-2.C), the flow of execution is determined by external events.
- The program registers 'event listeners' or 'event handlers' for specific events (e.g., onButtonClick, onTimerTick, onSensorThreshold).
- When an event occurs, the runtime triggers the corresponding event handler. If the user never clicks a button, that button's click handler never executes.
- This contrasts with classic sequential scripts that run strictly from start to finish without pausing for interactive input.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Strict linear execution describes simple sequential batch scripts, not event-driven interactive applications.
✗ **(C):** Compilers do not order execution alphabetically; procedures execute when invoked or triggered by events.
✗ **(D):** CPU temperature affects hardware thermal throttling, never the logical scheduling of event handler routines.

Question 23: Identifying Program Outputs

★★★ MANDATORY

Correct Answer: (B) — The electrical signal triggering the mechanical water pump valve to open and release water.

STEP-BY-STEP EXPLANATION

- Program outputs include data displayed to users, data written to files, and physical actuation signals sent to mechanical hardware (actuators).
- Activating the water pump valve converts software commands into physical work in the real world, functioning as an actuator output.
- The moisture sensor provides input, sunlight is an environmental condition, and the 25% threshold is an internal variable/constant.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The sensor probe captures data from the physical environment, making it an input.
✗ **(C):** Sunlight is an ambient natural environmental factor, not data produced by the program.
✗ **(D):** The 25% constant is an internal programmatic parameter used for conditional evaluation, not an output.

Question 24: Event Handler Pseudocode Analysis

★★★ MANDATORY

Correct Answer: (B) — Whenever the submit button is clicked while nameInputBox contains an empty string ("").

STEP-BY-STEP EXPLANATION

- Tracing event handler logic requires evaluating the trigger event and internal conditional branches.
- The handler is triggered when clickSubmitButton is clicked.
- The condition `IF (userName = "")` evaluates to true when the input box is blank.
- Inside that true branch, the code updates the error label and invokes `playSound("buzz.mp3")`, producing auditory output.
- If userName is not empty, the ELSE branch executes, which displays a welcome message and transitions screens without playing the buzz sound.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Entering characters sends execution to the ELSE branch, which does not play the sound.
✗ **(C):** The code is inside a click event listener; it only runs when the button is clicked, not in an uncontrolled loop.
✗ **(D):** Screen navigation occurs in the ELSE branch, where the sound is never triggered.

Question 25: Tactile/Haptic Outputs & Accessibility

★★★★★ VERY IMPORTANT

Correct Answer: (A) — It allows users who are visually impaired or in bright direct sunlight to receive physical confirmation that their transaction succeeded.

STEP-BY-STEP EXPLANATION

- Multi-modal outputs (visual display, auditory cues, and tactile/haptic vibrations) enhance accessibility and usability across diverse environments.
- Visually impaired users relying on screen readers or physical feedback can confirm critical actions (like financial transfers) without needing to inspect visual pixels.
- It also benefits sighted users in high-glare environments where screen visibility is compromised.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Haptic feedback is a user interface output mechanism; it has no effect on banking ledger account limits.
✗ **(C):** Visual rendering and haptic actuation operate via independent hardware subsystems and do not require each other.
✗ **(D):** Adding haptic feedback for transactional confirmation does not reclassify financial utility software as an entertainment game.

Question 26: Sequential vs. Event-Driven Execution

★★★ MANDATORY

Correct Answer: (B) — An interactive touchscreen kiosk at an airport where travelers tap buttons to look up flight gates, search food options, or call for wheelchair assistance.

STEP-BY-STEP EXPLANATION

- Event-driven programming is ideal for interactive user interfaces where the program must idle until a user initiates unpredictable actions (tapping buttons, swiping, selecting options).
- An airport kiosk cannot predict which button a traveler will press or when they will approach the screen; it must listen for user touch events asynchronously.
- Batch data processing, file conversions, and mathematical factorial calculations are classic sequential batch programs that execute in predetermined order without human intervention.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Batch log analysis processes existing data sequentially from start to finish without pausing for interactive events.
✗ **(C):** Computing a factorial is a purely algorithmic sequential calculation.
✗ **(D):** File format conversion runs linearly through the file until transcoding is complete.

Question 27: Input Validation as a Defense

★★ IMPORTANT

Correct Answer: (A) — A runtime error (exception) caused by an invalid string-to-integer conversion format.

STEP-BY-STEP EXPLANATION

- Runtime errors (exceptions) occur while the program is actively executing when an operation cannot be completed successfully.
- Passing non-numeric alphabetic text ('twenty-five') to an integer conversion routine causes the runtime environment to throw a NumberFormatException or ValueError.
- Input validation checks whether input data conforms to expected formats (e.g., digits only) before processing, preventing runtime crashes.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** The source code syntax itself is valid; the error happens dynamically during execution when invalid data is supplied.
✗ **(C):** Overflow occurs when a numeric value exceeds the maximum representable integer limit; 'twenty-five' is a parsing failure, not an integer overflow.
✗ **(D):** Software parsing errors have zero capacity to cause physical hardware fires or keyboard damage.

Question 28: Event-Driven Shared State Conflicts

★★★★★ VERY IMPORTANT

Correct Answer: (B) — The event queue (or event loop) processes incoming events sequentially in strict arrival order, timestamping each event to resolve concurrency.

STEP-BY-STEP EXPLANATION

- In event-driven architectures, asynchronous events from users or network sockets are placed into an 'event queue'.
- The runtime event loop dequeues and processes events sequentially in order of receipt, using high-precision timestamps to establish precedence.
- This deterministic queue prevents race conditions and ensures fair, orderly resolution of concurrent user interactions.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Username length has no connection to fair game timing and constitutes an arbitrary, flawed mechanism.
✗ **(C):** Modern event loops are explicitly designed to handle high volumes of concurrent events without crashing.
✗ **(D):** Operating systems do not terminate displays in response to simultaneous input events.

Question 29: Multi-Source Sensor Inputs

★★★ MANDATORY

Correct Answer: (A) — Sensor fusion.

STEP-BY-STEP EXPLANATION

- Sensor fusion is the process of combining sensory data from multiple disparate sources so that the resulting information has less uncertainty than would be possible when these sources were used individually.
- In robotics and autonomous vehicles, no single sensor is perfectly accurate (e.g., GPS drifts indoors, cameras struggle in fog); fusing multiple sensor inputs produces robust navigational outputs.
- Lossy compression reduces file size by discarding data, and phishing is a cybersecurity deception attack.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** 'Binary fragmentation' is not a computing term for multi-sensor integration.
✗ **(C):** Lossy compression refers to irreversible reduction in multimedia data fidelity.
✗ **(D):** Phishing is a social engineering attack aimed at stealing credentials, completely unrelated to drone sensors.

Question 30: Output Persistence & File Storage

★★ IMPORTANT

Correct Answer: (B) — To non-volatile secondary storage (such as a hard drive, solid-state drive, or cloud database file).

STEP-BY-STEP EXPLANATION

- RAM (Random Access Memory) and CPU registers are volatile memory; all data stored in them is erased when the program terminates or the machine loses power.
- Non-volatile storage (SSDs, magnetic disks, flash memory, cloud databases) persists data permanently even when powered off.
- To save outputs for future sessions, programs must serialize and write data to persistent files or databases.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** RAM is volatile and loses all stored values upon program exit or reboot.
✗ **(C):** CPU registers hold immediate operational operands for fractions of a nanosecond and are volatile.
✗ **(D):** Display buffers only hold current frame pixel colors and refresh 60–120 times per second without persistent file storage.

Question 31: Iterative vs. Incremental Development

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Incremental development, because the program is broken into smaller manageable parts, each of which is built, tested, and added to the existing system sequentially.

STEP-BY-STEP EXPLANATION

- Incremental development (CRD-2.E) involves breaking a software project into small, functional slices (increments).
- Each increment adds a distinct piece of working functionality (e.g., login -> grade entry -> report generation). Each piece is coded and verified before moving to the next.
- Iterative development involves revisiting and refining previous work through repeated cycles of prototyping and feedback.
- Combining incremental and iterative development allows teams to catch defects early, manage complexity, and respond flexibly to evolving requirements.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** The traditional waterfall model requires all requirements and design to be completely finished for the entire system before any coding begins, rather than building modular increments.

✗ **(C):** Incremental engineering is a deliberate, highly organized professional practice, not an unplanned accident.

✗ **(D):** 'Binary refactoring' is an invented term that does not describe modular software development phases.

Question 32: The Role of Prototyping

★★★★★ VERY IMPORTANT

Correct Answer: (B) — To explore user interface ideas, uncover usability flaws, and validate user workflows rapidly without investing heavy engineering resources into complex code.

STEP-BY-STEP EXPLANATION

- Prototyping is an exploratory phase in program design and development.
- Prototypes (ranging from paper wireframes to interactive digital mockups) allow designers and stakeholders to visualize navigation flows, test user intuition, and gather feedback quickly.
- Fixing a design flaw in a visual prototype takes minutes; fixing the same flaw after writing thousands of lines of full-stack code can take weeks of costly refactoring.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Wireframes do not interact with physical smartphone GPS hardware chips or vehicle speeds.

✗ **(C):** A prototype is a preliminary design model; production software still requires backend algorithms, databases, and network code.

✗ **(D):** Prototypes are built for engineering validation and user testing, not legal copyright registration.

Question 33: Iterative Refinement Cycles

★★★ MANDATORY

Correct Answer: (B) — Investigate user needs -> Design algorithm -> Prototype implementation -> Test with users and collect feedback -> Refine design based on results.

STEP-BY-STEP EXPLANATION

- College Board Learning Objective CRD-2.E defines the software development process as an ongoing cycle of: investigating/reflecting, designing, prototyping, and testing.
- In iterative development, feedback from testing directly informs the next cycle of refinement and design adjustments.
- This continuous loop ensures that the final software product meets quality, performance, and user satisfaction goals.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Deploying code without testing or reading user feedback is the antithesis of iterative development.

✗ **(C):** Compiling and minifying are build-pipeline steps, not a user-centered development process.

✗ **(D):** Shredding paper and retyping code from memory is an absurd workflow that produces no iterative refinement.

Question 34: Investigating & Reflecting Phase

★★★ MANDATORY

Correct Answer: (A) — Interview elementary school math teachers and observe young students solving fraction problems to identify common misconceptions and learning roadblocks.

STEP-BY-STEP EXPLANATION

- The investigation and reflection phase focuses on understanding the problem space, target audience, and functional requirements before building.
- By interviewing teachers and observing elementary students, the developer uncovers specific pedagogical needs (e.g., visual pizza slices vs. number lines) that guide effective software design.
- Starting development with deep user empathy and domain research prevents building a game that is too difficult or pedagogically ineffective.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Corporate tax registration is a legal business administration activity, not software investigation.
✗ **(C):** Purchasing server hardware before even designing the game is premature and unnecessary for an initial educational prototype.
✗ **(D):** Drive encryption is a local security hygiene practice unrelated to discovering educational fraction requirements.

Question 35: Design Tools: Storyboards & Flowcharts

★★ IMPORTANT

Correct Answer: (A) — They allow developers to plan and verify algorithmic logic conceptually without getting bogged down in language-specific syntax rules.

STEP-BY-STEP EXPLANATION

- Flowcharts and pseudocode are language-independent modeling tools used during the design phase.
- They allow developers to think through branching logic, loops, and data flows clearly and collaborate with non-technical stakeholders without worrying about semicolons, braces, or indentation syntax.
- Verifying logic in a flowchart helps eliminate conceptual flaws before coding begins.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Flowcharts are visual documentation on paper or screens; they cannot be inserted into CPU sockets.
✗ **(C):** Operating systems execute compiled binaries or bytecode; they do not require webcam scans of flowchart drawings.
✗ **(D):** Visual design models have no physical effect on Wi-Fi router electromagnetic frequencies or bandwidth.

Question 36: Benefits of Incremental Testing

★★★★★ VERY IMPORTANT

Correct Answer: (A) — When a bug occurs, the programmer knows the defect is almost certainly located in the most recently added function, making debugging drastically faster and easier.

STEP-BY-STEP EXPLANATION

- Incremental testing isolates defects immediately. If function A, B, and C worked perfectly, and the system fails immediately after adding function D, the developer can pinpoint function D (or its interface with C) as the source of the bug.
- Conversely, testing all 20 functions simultaneously for the first time creates a chaotic debugging nightmare where multiple interacting bugs mask one another.
- College Board CRD-2.E emphasizes that incremental development accompanied by frequent testing dramatically reduces debugging time.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Unit testing functions incrementally validates internal logic, but integration and end-user testing are still essential.
✗ **(C):** Hard drives do not reverse direction during unit testing, and software does not experience mechanical wear.
✗ **(D):** Functions designed to return values must still include return statements regardless of when they are tested.

Question 37: User Stories & Acceptance Criteria

★★★ MANDATORY

Correct Answer: (A) — 'As a library member, I want to scan a book's barcode with my camera so that I can instantly check if the book is available in our catalog.'

STEP-BY-STEP EXPLANATION

- A well-structured user story identifies a specific human role ('library member'), a concrete action ('scan a barcode with camera'), and a clear functional benefit ('instantly check catalog availability').
- User stories keep the development team laser-focused on real user needs and provide clear acceptance criteria for testing.
- Hardware power needs, bad coding habits, and system crashes are not user stories.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Silicon transistors are hardware components, not human software end users.
✗ **(C):** Writing unmaintainable spaghetti code actively damages software quality and harms teams.
✗ **(D):** Software crashes and unhandled exceptions are severe defects, not purposeful user goals.

Question 38: Refactoring in Iterative Development

★★ IMPORTANT

Correct Answer: (A) — Code refactoring.

STEP-BY-STEP EXPLANATION

- Code refactoring is the process of restructuring existing computer code without changing its external behavior or functionality.
- Extracting duplicated logic into a centralized helper function reduces redundancy (DRY principle: Don't Repeat Yourself), improves readability, and makes future maintenance easier.
- If the conversion formula ever needs updating, the developer only modifies one function instead of searching through multiple duplicate copies.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Phishing is a fraudulent cyberattack designed to trick individuals into disclosing sensitive credentials.
✗ **(C):** Denial of Service (DoS) is an attack aimed at overwhelming server network bandwidth.
✗ **(D):** Overclocking is increasing a hardware processor's clock speed beyond manufacturer ratings.

Question 39: Agile vs. Waterfall in Creative Development

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Because user needs, market conditions, and technical discoveries evolve rapidly, and iterative frameworks allow teams to pivot and adapt features based on ongoing feedback.

STEP-BY-STEP EXPLANATION

- Iterative/agile frameworks acknowledge that software requirements are rarely static. As users interact with early prototypes, their expectations and desires become clearer.
- Working in short iterative cycles allows teams to validate assumptions, incorporate user feedback, and pivot direction before spending months on unwanted features.
- The waterfall model's rigid linear phases make late-stage modifications prohibitively expensive and risky.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Agile frameworks emphasize continuous, rigorous automated and user testing throughout every sprint.
✗ **(C):** The waterfall model is an engineering process framework used across many languages; it has nothing to do with punch cards.
✗ **(D):** All software engineering requires human time, computational resources, and electrical energy regardless of methodology.

Question 40: Documenting Design Decisions

★★★ MANDATORY

Correct Answer: (A) — It ensures that current and future team members understand *why* the technical choice was made, preventing future developers from accidentally undoing the architectural foundation.

STEP-BY-STEP EXPLANATION

- Documenting design decisions and architectural trade-offs provides vital institutional knowledge for ongoing maintenance and onboarding.
- When new developers join the team months or years later, they can review the rationale behind choosing WebSockets (low latency, bi-directional communication) over polling, avoiding costly misinformed redesigns.
- Documentation is an essential practice under College Board Learning Objective CRD-2.G.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Natural alphabets are public domain and cannot be patented by software developers.
✗ **(C):** Written documentation does not physically alter an end user's ISP broadband connection speed.
✗ **(D):** Documentation files have no effect on client web browser rendering engines or CSS parsers.

Question 41: Syntax vs. Runtime vs. Logic Error Definitions

★★★★★ VERY IMPORTANT

Correct Answer: (C) — A logic error, because the program executes completely without crashing but produces an incorrect result due to an algorithmic mistake (using addition instead of multiplication).

STEP-BY-STEP EXPLANATION

- A *logic error* (CRD-2.I) occurs when an algorithm is flawed. The program conforms to all language grammar rules (so it compiles) and performs valid arithmetic operations (so it doesn't crash), but it computes the wrong result.
- Here, adding length and width (`5 + 4 = 9`) instead of multiplying (`5 * 4 = 20`) is a classic logic error.
- Syntax errors prevent compilation; runtime errors cause unexpected termination; overflow errors occur when numbers exceed bit representations.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Addition is syntactically valid in all programming languages; there is no grammar violation.
✗ **(B):** The program executed to completion without any crash or operating system abort; runtime errors terminate execution prematurely.
✗ **(D):** Modern systems store integers well into the billions; 20 fits easily in 8, 16, 32, or 64 bits.

Question 42: Identifying Syntax Errors

★★★★★ VERY IMPORTANT

Correct Answer: (A) — A syntax error, because the code violates the grammatical structural rules of the programming language.

STEP-BY-STEP EXPLANATION

- A *syntax error* represents a violation of the programming language's strict grammar and punctuation rules (such as a missing closing parenthesis `)` or mismatched quotes).
- Syntax errors are detected during parsing/compilation before any instructions are executed, preventing the program from running at all.
- Once the student inserts the missing `)`, the code becomes syntactically valid.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** The program never ran, so no logical output was produced; this is a syntactic grammar failure.
✗ **(C):** Zero-division is a mathematical runtime error that happens during active computation; this code was halted before running.
✗ **(D):** Local source code parsing errors have nothing to do with network hardware or wireless packets.

Question 43: Identifying Runtime Errors (Exceptions)

★★★★★ VERY IMPORTANT

Correct Answer: (B) — A runtime error occurs (such as an 'IndexOutOfBoundsException' or 'IndexError') because the program attempts to access an element at an index that does not exist in the list.

STEP-BY-STEP EXPLANATION

- The list `numbersList` contains 3 elements (indices 1, 2, and 3).
- Attempting to read `numbersList[5]` tries to access memory beyond the valid boundaries of the array/list structure.
- In standard AP CSP pseudocode and most modern languages, accessing an index that does not exist triggers an immediate *runtime error* (exception) that halts execution unless explicitly handled.
- The syntax itself is completely valid, so it is not a syntax error.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The list only has 3 items; it cannot fabricate a value of 50 out of nowhere.
✗ **(C):** The syntax conforms to language rules; the error is dynamic and only occurs at runtime when the invalid index is evaluated.
✗ **(D):** Standard list access operations do not automatically allocate or auto-expand list capacity during read operations.

Question 44: Divide-by-Zero Runtime Error

★★★ MANDATORY

Correct Answer: (A) — A runtime error will occur because division by zero is mathematically undefined and cannot be executed by the computer processor.

STEP-BY-STEP EXPLANATION

- Division by zero is an undefined mathematical operation.
- When integer division by zero is attempted during execution, the processor or runtime environment generates a *runtime error* (e.g., `ZeroDivisionError` or `ArithmeticException`), immediately terminating the program.
- Robust programs prevent this runtime error by including input validation or an `IF (studentCount > 0)` check before performing the division.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** In standard integer arithmetic, computers do not quietly proceed with infinity; they throw a runtime exception.
✗ **(C):** The division operator `/` is valid syntax; the error only manifests when the denominator evaluates to zero during execution.
✗ **(D):** Operating systems protect hard drives; software arithmetic exceptions never cause drive reformatting.

Question 45: Overflow Errors

★★★★★ VERY IMPORTANT

Correct Answer: (B) — An overflow error, because the resulting integer value (256) requires 9 bits (100000000), which exceeds the fixed 8-bit memory allocation.

STEP-BY-STEP EXPLANATION

- An *overflow error* occurs when a calculated integer exceeds the maximum finite number of bits allocated to store it.
- In an 8-bit unsigned system, the maximum representable integer is $2^8 - 1 = 255$.
- Adding 1 requires a 9th bit (100000000_2). Because only 8 bits are retained, the 9th bit overflows, typically wrapping the stored value back around to 0 (00000000_2).
- College Board Big Idea 2 and Big Idea 1 highlight overflow as a critical computational boundary error.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Adding 1 to a variable is valid syntax; the limitation is hardware memory representation.
✗ **(C):** Lossy compression refers to throwing away perceptual detail in media files (like JPEG or MP3), not integer arithmetic overflow.
✗ **(D):** The error is rooted in the digital bit representation of the counter in software/hardware, not in the physical door sensor.

Question 46: Off-by-One Logic Error

★★★ MANDATORY

Correct Answer: (B) — Outputs: 1, 2, 3, 4 (stops before displaying 5) due to an off-by-one logic error.

STEP-BY-STEP EXPLANATION

- Tracing the loop: • count = 1: displays 1, count becomes 2. • count = 2: displays 2, count becomes 3. • count = 3: displays 3, count becomes 4. • count = 4: displays 4, count becomes 5. • Next check: `REPEAT UNTIL (count = 5)` evaluates to TRUE, so the loop terminates immediately *before* executing the body for 5!
- The program outputs: 1, 2, 3, 4.
- Because the programmer intended to include 5, this is a classic *off-by-one logic error*. Changing the condition to `count > 5` resolves the issue.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** The loop terminates the instant count reaches 5, so 5 is never displayed.

✗ **(C):** The syntax of `REPEAT UNTIL` is perfectly valid AP CSP pseudocode.

✗ **(D):** The loop terminates deterministically when count equals 5; it is not infinite.

Question 47: Type Mismatch Logic/Runtime Error

★★ IMPORTANT

Correct Answer: (A) — Displays "1020" because the variables contain string text rather than numeric integers, concatenating the characters instead of performing mathematical addition.

STEP-BY-STEP EXPLANATION

- Quotation marks indicate string literals (`"10"` and `"20"`).
- When the `+` operator is applied to two strings, it performs concatenation (joining them end-to-end), resulting in `"1020"`.
- If the programmer intended to calculate a sum (`10 + 20 = 30`), this constitutes a logic error stemming from using string data types instead of numeric integer types.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Computers respect variable data types; they do not automatically convert string literals to integers without explicit conversion functions.

✗ **(C):** Screens display strings and numbers routinely; no error is thrown by displaying a string.

✗ **(D):** Concatenation joins characters together; it does not erase digits or return 0.

Question 48: Boolean Operator Logic Error

★★★ MANDATORY

Correct Answer: (C) — A rider who is 40 inches tall and 11 years old.

STEP-BY-STEP EXPLANATION

- The program used `OR` instead of `AND`.
- For a rider with `height = 40` (too short!) and `age = 11` (old enough): • `height >= 48` is FALSE. • `age >= 10` is TRUE. • `FALSE OR TRUE` evaluates to TRUE! • The program displays 'Eligible to ride', which violates the safety requirement that riders must satisfy *both* conditions.
- This test case exposes the boolean operator logic error.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Both conditions are true (52 >= 48 and 12 >= 10), so both `AND` and `OR` would yield the same result ('Eligible').

✗ **(B):** Both conditions are false (36 < 48 and 6 < 10), so both `AND` and `OR` yield 'Not eligible'.

✗ **(D):** Both conditions are easily met, producing the correct outcome regardless of the `OR` bug.

Question 49: Floating-Point Round-Off Error

★★★★★ VERY IMPORTANT

Correct Answer: (B) — A round-off error caused by the fact that certain decimal fractions (like 0.1) cannot be represented with exact precision in finite binary floating-point representation.

STEP-BY-STEP EXPLANATION

- Just as 1/3 cannot be represented with finite decimal digits (0.3333...), fractions like 0.1 and 0.2 cannot be represented with finite binary bits (they repeat infinitely in binary: 0.0001100110011...).
- Because computers store numbers using a fixed number of bits (such as 64-bit IEEE 754 floating-point), the trailing bits are truncated or rounded.
- This slight inaccuracy is called a *round-off error*. Over millions of calculations, round-off errors can accumulate, which is why software engineers use tolerance thresholds (e.g., `abs(a - b) < 0.00001`) instead of direct equality `==` when comparing floats.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Decimal points are universally valid syntax in high-level programming languages.
✗ **(C):** Floating-point imprecision is a fundamental mathematical property of binary representations, not malware.
✗ **(D):** Hardware standards like IEEE 754 are engineered for optimal speed and precision balance, not homework obstruction.

Question 50: Compile-Time vs. Run-Time Error Detection

★★ IMPORTANT

Correct Answer: (A) — A missing closing quotation mark on a string literal on line 42.

STEP-BY-STEP EXPLANATION

- A missing closing quote is a *syntax error*.
- Compilers and syntax parsers analyze code grammar prior to execution. If a syntax error is present, parsing fails immediately, and the program cannot even be converted into an executable.
- Divide-by-zero, out-of-bounds indices, and wrong tax rate constants all pass syntax validation and are only discovered during execution or user testing.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** User input is not evaluated until runtime, so divide-by-zero cannot be caught during initial compilation.
✗ **(C):** Dynamic array access depends on runtime user interactions and cannot be completely proven at compile time.
✗ **(D):** The compiler cannot know the developer's real-world intention regarding tax rates; 0.05 is syntactically valid code.

Question 51: Tracing Variable Values with a Trace Table

★★★★★ VERY IMPORTANT

Correct Answer: (B) — x = 7, y = 3

STEP-BY-STEP EXPLANATION

- We trace each variable step-by-step using a trace table: • Step 1: `x <- 3` (x is 3) • Step 2: `y <- 7` (y is 7) • Step 3: `x <- x + y` -> `x <- 3 + 7 = 10` (x is now 10, y is 7) • Step 4: `y <- x - y` -> `y <- 10 - 7 = 3` (x is 10, y is now 3) • Step 5: `x <- x - y` -> `x <- 10 - 3 = 7` (x is now 7, y is 3)
- This sequence is a famous classical algorithmic technique for swapping the values of two variables in-place without requiring a temporary third variable.
- The final values are x = 7 and y = 3.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** These are the initial values before the swapping arithmetic occurs.
✗ **(C):** Step 4 and Step 5 subtract the values, so both variables do not remain at 10.
✗ **(D):** The operations perform systematic value exchange, not zeroing.

Question 52: Using Print Statements for Debugging

★★★★★ VERY IMPORTANT

Correct Answer: (A) — To inspect the internal state of variables during execution to pinpoint exactly which statement causes the data to deviate from expected values.

STEP-BY-STEP EXPLANATION

- Inserting temporary print/display statements (often called 'print debugging' or 'trace output') is one of the most accessible and effective debugging strategies (CRD-2.J).
- By observing how variables change at different checkpoints during execution, the programmer can identify the exact line of code where the calculation first goes wrong.
- Once the defect is identified and fixed, the temporary print statements are removed before deployment.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Displaying text involves I/O operations, which slightly slows down execution rather than speeding up the CPU.
✗ **(C):** Diagnostic debug statements are temporary internal tools and should never be exposed to final end customers.
✗ **(D):** Print statements output text to the console; they provide no cryptographic protection or anti-tamper security.

Question 53: Boundary Value Testing

★★★★★ VERY IMPORTANT

Correct Answer: (B) — 12, 13, 64, and 65.

STEP-BY-STEP EXPLANATION

- Boundary value testing focuses on the edges of input domains where relational operators (`<`, `<=`, `>`, `>=`) frequently harbor off-by-one errors.
- The transition boundaries in this pricing rule are: • Child to Adult: 12 (child) vs. 13 (adult) • Adult to Senior: 64 (adult) vs. 65 (senior)
- Testing the exact transition values (12, 13, 64, 65) confirms that the boundary inequalities are strictly implemented according to specifications.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** 25 and 40 are interior values in the adult bracket and will not expose boundary off-by-one errors at 12 or 65.
✗ **(C):** These are extreme outliers that all fall into the same senior category without testing the child or adult boundaries.
✗ **(D):** Negative numbers test invalid input handling, but fail to test the critical legal transition boundaries.

Question 54: Trace Table Analysis with Loops

★★★ MANDATORY

Correct Answer: (B) — 12

STEP-BY-STEP EXPLANATION

- We trace the loop across its 3 iterations: • Initially: `total = 0`, `k = 1` • Iteration 1: - `total <- 0 + (1 * 2) = 2` - `k <- 1 + 1 = 2` • Iteration 2: - `total <- 2 + (2 * 2) = 2 + 4 = 6` - `k <- 2 + 1 = 3` • Iteration 3: - `total <- 6 + (3 * 2) = 6 + 6 = 12` - `k <- 3 + 1 = 4` • Loop ends. `DISPLAY(total)` outputs 12.
- Systematic trace tables prevent mental arithmetic errors when working with accumulating variables.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** 6 is the intermediate value of total after only 2 iterations.
✗ **(C):** 14 is an arithmetic miscalculation adding an incorrect product.
✗ **(D):** 18 would require a 4th iteration with k = 4 (12 + 8 = 20) or miscalculated multipliers.

Question 55: Using Breakpoints in a Debugger

★★★ MANDATORY

Correct Answer: (B) — Program execution pauses immediately before line 45 executes, allowing the developer to inspect current variable values in memory and step forward one instruction at a time.

STEP-BY-STEP EXPLANATION

- A breakpoint is a designated stopping point placed by a programmer inside source code.
- When running in debug mode, the IDE executes normally until it hits the breakpoint, where it freezes execution state.
- The developer can then inspect the exact contents of variables, evaluate expressions, and 'step over' or 'step into' lines to observe execution step-by-step.
- Breakpoints eliminate the need to manually write dozens of temporary print statements.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Breakpoints are non-destructive markers; they never delete or alter source code.
✗ **(C):** Debugging tools pause software execution inside the IDE; they do not alter hardware screen power.
✗ **(D):** Execution always begins at the designated entry point and runs lines 1 through 44 normally before pausing at line 45.

Question 56: Test Case Coverage & Edge Cases

★★ IMPORTANT

Correct Answer: (B) — An empty list `[]` containing zero words.

STEP-BY-STEP EXPLANATION

- An 'edge case' (or boundary case) is an unusual or extreme operating condition at the extreme limits of the input domain.
- Passing an empty list `[]` tests how the algorithm handles having no elements to inspect (e.g., does it crash with an IndexError, return null, or throw a clean error?).
- Robust programs explicitly account for edge cases like empty lists, single-item lists, and duplicate ties.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** 10,000 words of equal length tests performance or tie-breaking, but is not the minimal boundary edge case.
✗ **(C):** Fonts are visual presentation formatting; string lists in memory only store character data.
✗ **(D):** Physical paper is not a computational input to an in-memory data structure.

Question 57: Isolating Defects by Commenting Code

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Code isolation: by systematically disabling sections of code, the developer narrows down the search space to confirm that the bug resides within the disabled block.

STEP-BY-STEP EXPLANATION

- Isolating code by commenting out sections (or using binary search debugging) helps locate elusive bugs in large files.
- If disabling lines 200–350 resolves the crash, the bug (or an incompatible call) is mathematically isolated to that specific block.
- The developer can then uncomment smaller sub-segments within that range until the exact offending line is revealed.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Commenting out code tests execution dependency, not human linguistic origins.
✗ **(C):** Software comments have no impact on electrical power supplies or physical grid reliability.
✗ **(D):** Commented code is completely ignored by the compiler and is not rewritten or executed.

Question 58: Automated Unit Testing

★★★ MANDATORY

Correct Answer: (A) — To detect 'regression bugs'—situations where newly added features or bug fixes accidentally break existing, previously working features.

STEP-BY-STEP EXPLANATION

- Automated regression testing is crucial in software maintenance.
- When a developer modifies code to add a feature or fix a defect, there is always a risk that the change inadvertently damages another part of the system (a regression).
- Running a comprehensive suite of unit tests automatically flags regressions immediately, guaranteeing that existing functionality remains intact.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** There is no legal quota on line counts; efficient software often strives for concise, modular code.
✗ **(C):** Unit tests validate functional code logic, not physical display hardware brands.
✗ **(D):** Automated testing supports team collaboration by building trust; it does not eliminate human communication.

Question 59: Trace Table with Nested Conditions

★★ IMPORTANT

Correct Answer: (A) — 4

STEP-BY-STEP EXPLANATION

- Given the initial values: a = 4 and b = 8.
- The outer condition `IF (a > 2)` evaluates to TRUE since 4 > 2.
- Execution enters the outer IF block and evaluates the nested condition `IF (b < 5)`.
- Since 8 < 5 evaluates to FALSE, execution branches directly into the inner ELSE block.
- The statement `c <- b - a` executes: `c <- 8 - 4`, assigning 4 to variable c.
- The outer ELSE block is skipped, and `DISPLAY(c)` outputs 4.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** 12 would occur if `b < 5` were true (`a + b = 4 + 8 = 12`), but 8 is not less than 5.
✗ **(C):** 32 would occur if `a > 2` were false (`a * b = 4 * 8 = 32`), but 4 is greater than 2.
✗ **(D):** None of the branches produce a value of 0.

Question 60: Systematic vs. Trial-and-Error Debugging

★★★★★ VERY IMPORTANT

Correct Answer: (A) — It wastes time, rarely addresses the root cause of the bug, and often introduces new, hidden defects that make the program even harder to fix.

STEP-BY-STEP EXPLANATION

- Systematic debugging requires forming a hypothesis, tracing code, and making deliberate, controlled modifications.
- Random trial-and-error changes mask the true underlying logic flaw and often introduce subtle secondary bugs.
- Even if a random change appears to work for one test case, it usually breaks multiple other scenarios because the developer does not understand *why* the change affected the outcome.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Programmers can modify their own source code as many times as needed without legal restriction.
✗ **(C):** Compilers convert code to machine binary during build phases, not when characters are typed.
✗ **(D):** Typing random characters in a text editor does not cause hardware thermal overload.

Question 61: Purpose of Program Documentation

★★★★★ VERY IMPORTANT

Correct Answer: (B) — To explain the design, purpose, parameters, and behavior of the code to current and future programmers, enhancing readability and maintainability.

STEP-BY-STEP EXPLANATION

- Program documentation (CRD-2.G) is written for human understanding, not for the computer compiler.
- It describes what code does, how algorithms work, what parameters functions expect, and what values are returned.
- Proper documentation allows new developers to onboard quickly, assists original authors when returning to code months later, and significantly reduces the cost of maintaining and expanding software over time.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Comments and documentation are ignored by compilers or stripped during compilation; they have zero impact on CPU execution speed.
✗ **(C):** Syntax errors occur when code grammar is broken; documentation comments cannot fix or prevent syntax errors in executable code.
✗ **(D):** Computers do not require English explanations; CPUs execute raw binary instructions directly.

Question 62: How Compilers Treat Comments

★★★★★ VERY IMPORTANT

Correct Answer: (A) — It completely ignores the comments during translation, so they consume zero CPU execution time and do not alter the program's output.

STEP-BY-STEP EXPLANATION

- Comments (e.g., `//` or `/* ... */`) are lexical tokens designed exclusively for human readers.
- Compilers and interpreters strip comments out during the preprocessing or parsing phase.
- Therefore, comments have absolutely no effect on runtime performance, memory overhead, or programmatic output.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Comments are explicitly skipped; they are never parsed or executed as commands.
✗ **(C):** Programs do not pause or lag when encountering comments because comments are removed before runtime.
✗ **(D):** Comments are text in the source code; they do not trigger audio synthesis.

Question 63: Self-Documenting Code & Identifier Naming

★★★ MANDATORY

Correct Answer: (B) — Implementation 2 uses self-documenting, meaningful identifier names that make the code self-explanatory, readable, and less prone to misunderstandings during maintenance.

STEP-BY-STEP EXPLANATION

- Self-documenting code uses descriptive, meaningful variable and procedure names (like `salesTax` and `totalWithTax` rather than `x` and `y`).
- Anyone reading Implementation 2 instantly understands the financial meaning of each variable without having to guess or cross-reference external notes.
- Descriptive naming dramatically reduces bugs during maintenance and refactoring.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Variable names are mapped to memory addresses during compilation; meaningful names execute at the exact same speed as single-letter names.
✗ **(C):** Single-letter identifiers like `x` and `y` are syntactically valid in all languages, even though they represent poor practice for business logic.
✗ **(D):** Longer variable names in source code slightly increase text file size before compilation, not the other way around.

Question 64: Documenting Preconditions & Postconditions

★★★ MANDATORY

Correct Answer: (A) — To state the assumptions or input requirements that must be true *before* the function is called for it to execute correctly.

STEP-BY-STEP EXPLANATION

- A *precondition* defines the constraints and requirements that must be satisfied by caller arguments before invoking the procedure.
- Here, the precondition `n >= 0` warns developers that the square root function cannot accept negative numbers (which would produce imaginary numbers or domain errors).
- A *postcondition* describes the guaranteed state or return value after the procedure executes successfully.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Preconditions define technical input constraints, not team contributor rosters.
✗ **(C):** Preconditions are documentation contracts, not hardware reboot commands.
✗ **(D):** The year 2038 problem involves 32-bit Unix timestamps, which is unrelated to square root preconditions.

Question 65: Modularity & Procedural Abstraction

★★ IMPORTANT

Correct Answer: (A) — It allows individual components to be developed, tested, and debugged independently, reducing overall system complexity.

STEP-BY-STEP EXPLANATION

- Modularity and procedural abstraction (CRD-2.H) break a large program into discrete, manageable sub-problems.
- Each module has a focused responsibility (e.g., collision detection). If physics behavior is glitchy, the developer knows exactly which procedure to debug without sifting through rendering code.
- Modules can also be reused across different projects.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Modularity organizes software structure; it cannot supply missing physical hardware like a GPU.
✗ **(C):** Modular code organization has no bearing on in-game win/loss mechanics.
✗ **(D):** Modular procedures routinely contain loops, conditions, and calculations as needed.

Question 66: Consequences of Outdated Documentation

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Documentation must be updated in tandem with code changes; out-of-date documentation is actively misleading and can cause serious logic errors.

STEP-BY-STEP EXPLANATION

- Code and documentation must remain synchronized. When code behavior changes, developers must update corresponding comments and specifications immediately.
- Outdated or contradictory documentation is often worse than no documentation at all because it misleads developers into making false assumptions, resulting in bugs.
- Maintaining documentation accuracy is an essential professional standard.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Documentation is crucial for maintainability; the solution is keeping it accurate, not banning comments.
✗ **(C):** New developers rely heavily on documentation to understand system architecture and avoid mistakes.
✗ **(D):** Source code comments are static text and are not automatically modified or deleted by compilers.

Question 67: When to Write Documentation

★★★ MANDATORY

Correct Answer: (A) — Throughout the development process—during planning, as code is being written, and whenever code is modified.

STEP-BY-STEP EXPLANATION

- Documentation is an integral, ongoing part of development, not an afterthought left for the end.
- Documenting requirements during planning, adding header comments as procedures are written, and updating notes during refactoring ensures documentation remains fresh and accurate.
- Waiting until the end of a project often leads to forgotten design rationales and incomplete documentation.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Documenting on the day of retirement is useless for maintenance during the product's active life.
✗ **(C):** Collaboration requires transparent, readable code and clear documentation, not secrecy.
✗ **(D):** Documentation is required for healthy, working software, not just failed test runs.

Question 68: Audience-Appropriate Documentation

★★ IMPORTANT

Correct Answer: (A) — Manual Alpha is technical documentation for software developers; Manual Beta is user documentation for everyday consumers.

STEP-BY-STEP EXPLANATION

- Documentation must be tailored to its target audience.
- Technical documentation (APIs, schemas, authentication protocols) is written for software engineers and systems integrators who write code to interface with the platform.
- End-user documentation (how-to guides, FAQ, user manuals) is written in non-technical, accessible language for consumers using the app's features.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** OAuth tokens and API endpoints are advanced computing concepts, not intended for young children.
✗ **(C):** These manuals document a software cloud application, not physical semiconductor manufacturing.
✗ **(D):** Manuals are written human-readable documents, not compiled kernel or driver binaries.

Question 69: API Documentation & Program Interfaces

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Abstraction, because the API documentation explains *how to use* the functions (inputs, outputs, and behaviors) without requiring the developer to understand the internal implementation details.

STEP-BY-STEP EXPLANATION

- Abstraction is a fundamental principle of computer science: managing complexity by hiding low-level details behind clean interfaces.
- An Application Programming Interface (API) exposes what functions exist, what arguments they take, and what they do.
- Developers can seamlessly integrate powerful mapping capabilities into their own applications by reading the API contract without needing to study the 50,000 lines of complex internal geometry algorithms.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** APIs represent procedural abstraction, not lossy compression of multimedia files.
✗ **(C):** Legitimate software libraries and API documentation are not phishing attacks.
✗ **(D):** Hardware overflow has nothing to do with reading software documentation.

Question 70: Code Refactoring to Improve Documentation

★★★ MANDATORY

Correct Answer: (A) — Maintainability and code readability.

STEP-BY-STEP EXPLANATION

- Renaming cryptic functions and variables to descriptive terms and adding explanatory comments directly elevates *maintainability* and *readability*.
- Future developers can read and modify the code with confidence, understanding immediately what parameters represent and what the function computes.
- Refactoring for clarity has no effect on physical hardware parameters like the speed of light, monitor resolution, or battery chemistry.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Software identifiers do not alter the physical speed of light in fiber optics.
✗ **(C):** Monitor resolution is determined by display hardware pixels, not variable naming.
✗ **(D):** Battery capacity is determined by chemical lithium-ion cells, completely independent of source code readability.

Question 71: Acknowledging External Code Authorship

★★★★★ VERY IMPORTANT

Correct Answer: (B) — Acknowledge the original author by including a comment in the program documentation citing the source, URL, author, and license of the imported code.

STEP-BY-STEP EXPLANATION

- Under College Board guidelines and professional computing ethics (CRD-2.H), using third-party code, open-source libraries, or external media is fully permitted provided that proper attribution is given.
- The student must include an explicit citation in program comments/documentation stating the original author, source URL, and license terms.
- Claiming someone else's work as your own or stripping attribution notices constitutes academic dishonesty and copyright infringement.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Falsely claiming authorship is plagiarism and violates academic integrity policies.
✗ **(C):** Obfuscating copied code to hide its origin is deceptive and unethical.
✗ **(D):** Stripping copyright headers violates open-source software license agreements.

Question 72: Open Source vs. Proprietary Software

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Open source software makes its underlying source code freely available for anyone to inspect, modify, and distribute, whereas proprietary software keeps its source code closed and restricted by commercial licensing.

STEP-BY-STEP EXPLANATION

- Open source software (e.g., Linux, Python, Blender) publishes its source code under open licenses (like MIT or Apache), allowing global communities to collaborate, audit security, and build upon it.
- Proprietary software (e.g., Microsoft Windows, Adobe Photoshop) treats source code as a trade secret and grants users only a restricted executable binary license.
- Open source software undergoes rigorous global peer review, often resulting in exceptional reliability and security.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Open source software is extensively tested by global developer communities and automated suites; proprietary software frequently contains bugs.
✗ **(C):** All modern computer software runs on silicon semiconductor processors regardless of license type.
✗ **(D):** The vast majority of open source software is distributed via public internet repositories like GitHub and package managers.

Question 73: Creative Commons Licensing Attributes

★★★ MANDATORY

Correct Answer: (A) — The student may use and modify the photos freely as long as they attribute the original creator and do not use the project for commercial profit.

STEP-BY-STEP EXPLANATION

- Creative Commons licenses use standardized modular terms: • 'BY' (Attribution) requires giving credit to the creator. • 'NC' (NonCommercial) restricts use to non-monetized, non-commercial purposes.
- Because the student's school project is non-commercial, they can use and adapt the images freely, provided they properly credit the author in the game's credits or documentation.
- Selling the images on merchandise would violate the 'NC' restriction.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Creative Commons licenses provide upfront public permissions without requiring individual upfront royalty payments.
✗ **(C):** Selling photos on t-shirts violates both the 'NC' (commercial restriction) and 'BY' (omitting attribution) clauses.
✗ **(D):** CC licenses explicitly grant permission to download and use the media according to the license conditions.

Question 74: Public Domain Dedication (CC0)

★★★★★ VERY IMPORTANT

Correct Answer: (A) — The creator has waived all copyright interests, meaning the developer can use, modify, and distribute the audio for both personal and commercial projects without permission or mandatory attribution.

STEP-BY-STEP EXPLANATION

- CC0 (Creative Commons Zero) is a universal public domain dedication tool.
- By applying CC0, the copyright owner relinquishes all copyright and related rights worldwide to the maximum extent permitted by law.
- Developers can use public domain assets in any software (open source or commercial) without legal risk, though professional ethics still encourages voluntary attribution.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** CC0 refers to zero copyright restrictions, not a zero-playback self-destruct timer.
✗ **(C):** Public domain dedication involves voluntary legal copyright waiver, not government equipment confiscation.
✗ **(D):** Public domain content has no time-of-day playback restrictions.

Question 75: Consequences of Software Plagiarism

★★★ MANDATORY

Correct Answer: (A) — It constitutes copyright infringement and fraud, subjecting the team to disqualification, reputational damage, and potential civil lawsuits from the original copyright holders.

STEP-BY-STEP EXPLANATION

- Copyright automatically protects original creative software code the moment it is fixed in a tangible medium of expression.
- Downloading someone else's proprietary or licensed code, removing attribution, and claiming it as your own for prize money is illegal copyright infringement and fraudulent misrepresentation.
- Such behavior results in immediate disqualification, legal liability, and severe professional consequences.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Plagiarism and theft of intellectual property violate professional ethics and are never celebrated in computing.
✗ **(C):** Patents are granted only for novel, non-obvious technical inventions through rigorous patent office examination, not for plagiarized submissions.
✗ **(D):** Publishing code online does not waive copyright; downloading code does not transfer intellectual property ownership.

Question 76: Software Patents vs. Copyrights

★★ IMPORTANT

Correct Answer: (A) — Copyright protects the specific creative expression of source code text, while a patent protects the underlying novel, useful, and non-obvious algorithmic method or system process.

STEP-BY-STEP EXPLANATION

- Copyright law automatically protects the literal creative expression of code (the specific text, syntax structure, and arrangement of written programs).
- Patent law, when granted by a government patent office, protects novel and inventive technical processes, algorithms, or machine architectures (the functional idea itself, regardless of how it is coded).
- Patent applications require extensive formal review and provide exclusive monopoly rights for a limited period (typically 20 years).

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Copyright applies broadly to literary and software works, and patents apply to inventions across industries.
✗ **(C):** Copyright typically lasts for the author's life plus 70 years; patents last approximately 20 years.
✗ **(D):** Copyright and patents are distinct legal doctrines with different standards of acquisition, scope, and enforcement.

Question 77: Citing Code on the AP CSP Create Performance Task

★★★★★ VERY IMPORTANT

Correct Answer: (A) — The student can use the external library, but in their written response explaining their program's algorithm, they must describe an algorithm that they personally developed (not the library's internal code) and must cite the external library in their documentation.

STEP-BY-STEP EXPLANATION

- College Board guidelines for the Create Performance Task explicitly state that students may utilize external libraries, APIs, and tools to support their program.
- However, when responding to written prompts regarding algorithmic logic, procedural abstraction, and data structures, students must select and explain algorithms that they *independently developed*.
- All external libraries, media, and third-party code must be clearly cited in project comments to maintain academic integrity.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** External libraries and modules are widely used and explicitly permitted on the Create Task.
✗ **(C):** Falsely claiming authorship of a third-party library is academic dishonesty and results in loss of points or score cancellation.
✗ **(D):** College Board does not charge licensing fees for student use of third-party libraries.

Question 78: Copyleft (GPL) Licensing Rules

★★★ MANDATORY

Correct Answer: (A) — They must make the source code of their entire image editing tool publicly available under the GPL license.

STEP-BY-STEP EXPLANATION

- The GNU General Public License (GPL) is known as a 'copyleft' or reciprocal license.
- Its core provision mandates that if you modify or incorporate GPL-licensed code into your own program and distribute it, your entire derivative program must also be licensed under the GPL and its source code made publicly accessible.
- This ensures that software freedom is preserved downstream for all future users.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Keeping the source code closed violates the reciprocal terms of the GPL, constituting copyright infringement.
✗ **(C):** Replacing algorithms with random numbers destroys program functionality and does not fulfill license obligations.
✗ **(D):** GPL compliance requires sharing source code digitally, not transferring physical computer hardware.

Question 79: Fair Use Doctrine in Computing

★★ IMPORTANT

Correct Answer: (A) — The Fair Use doctrine.

STEP-BY-STEP EXPLANATION

- The Fair Use doctrine (17 U.S.C. § 107) allows limited use of copyrighted material without permission from the copyright owner under specific circumstances.
- Key factors supporting fair use include: transformative purpose (commentary, criticism, education), non-commercial educational nature, using only a small necessary portion (2-second clips), and causing no substantial market harm to the original work.
- Fair use provides critical legal breathing room for scholarship, research, and journalism.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** The Digital Millennium Copyright Act (DMCA) criminalizes circumventing digital rights management (DRM), not an exemption for critique.
✗ **(C):** Open Hardware refers to sharing physical hardware schematics, not software copyright doctrine.
✗ **(D):** Blockchain consensus protocols manage decentralized ledgers, completely unrelated to copyright law.

Question 80: Ethical Sharing of Student Code

★★★ MANDATORY

Correct Answer: (B) — Classmate A talks through the algorithmic concept on a whiteboard, asks probing questions about Classmate B's logic, and guides them to discover the fix without sharing direct code.

STEP-BY-STEP EXPLANATION

- Ethical peer assistance focuses on conceptual understanding and problem-solving, not copying implementation.
- Whiteboarding the high-level algorithm, explaining concepts, and asking guiding questions helps the struggling student develop genuine problem-solving skills without violating honor codes.
- Sharing source code directly on individual assignments constitutes plagiarism and violates academic policies.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Sharing code for copy-pasting is academic dishonesty and plagiarism for both students.
✗ **(C):** Taking money to write another student's assignment is contract cheating.
✗ **(D):** Sabotaging a peer's repository is malicious and unethical.

Question 81: Full Lifecycle Defect Resolution

★★★★★ VERY IMPORTANT

Correct Answer: (A) — Regression testing: ensuring the new code modification fixes the cold-weather defect without breaking existing, previously verified bus tracking features.

STEP-BY-STEP EXPLANATION

- Running existing automated and manual test suites after applying a bug fix is known as *regression testing*.
- Its objective is to confirm that the patch successfully resolves the targeted defect while verifying that none of the previously working features (like GPS route tracking or stop notifications) were unintentionally broken.
- Regression testing is a cornerstone of professional software quality assurance.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Bus tracking software logs geolocation; it does not alter mechanical diesel engine horsepower.
✗ **(C):** Commit histories must be preserved for auditability and institutional memory, not erased.
✗ **(D):** Running regression tests executes existing test suites; it does not translate the codebase into assembly.

Question 82: Branch Coverage & Test Suite Completeness

★★★★★ VERY IMPORTANT

Correct Answer: (B) — 3 test cases.

STEP-BY-STEP EXPLANATION

- Branch coverage requires testing every reachable branch of conditional statements.
- This code contains three mutually exclusive operational paths: • Path 1: `isStudent = true` (discount = 0.15) • Path 2: `isStudent = false` AND `age >= 65` (discount = 0.10) • Path 3: `isStudent = false` AND `age < 65` (discount = 0.00)
- Therefore, exactly 3 carefully chosen test cases are needed to execute every branch: 1. `(isStudent: true, age: 20)` 2. `(isStudent: false, age: 70)` 3. `(isStudent: false, age: 30)`

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** A single test case can only execute one branch, leaving the other two paths completely untested.

✗ **(C):** While testing 10 cases is acceptable, the question asks for the *minimum* number required for complete branch coverage (3).

✗ **(D):** 100 test cases is vastly redundant for a simple 3-branch conditional tree.

Question 83: Multi-Error Diagnosis in Code Snippets

★★★ MANDATORY

Correct Answer: (B) — A runtime error (divide by zero exception) on Line 7.

STEP-BY-STEP EXPLANATION

- Tracing the code line by line: • Lines 1–6 execute properly: `sum` accumulates `4 + 8 + 12 = 24`.
- • Line 7 attempts integer division by zero (`sum / 0`). Because division by zero is mathematically undefined, the runtime environment throws an arithmetic exception and crashes immediately on Line 7.
- • Execution halts before Line 8 can display anything.
- This is a classic runtime error.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(A):** Square brackets `[...]` are standard AP CSP pseudocode notation for list initialization.

✗ **(C):** Line 5 performs standard accumulation inside a finite `FOR EACH` loop; it terminates after 3 iterations and is not infinite.

✗ **(D):** Displaying text to the console does not cause GPU hardware burnout.

Question 84: Merge Conflicts in Collaborative Development

★★★ MANDATORY

Correct Answer: (A) — The version control system detects a merge conflict on line 15 and pauses the merge, requiring the developers to communicate, inspect the differences, and choose or combine the correct code before committing.

STEP-BY-STEP EXPLANATION

- A *merge conflict* occurs when two developers modify the exact same lines of a file independently in different branches or commits.
- Modern version control systems (like Git) recognize that they cannot automatically determine which developer's version is correct.
- They flag the conflicting lines with conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) and require human developers to collaborate, resolve the discrepancy, and finalize the merge.
- This safety mechanism prevents accidental data loss.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Version control software manages source code text; it never deletes physical hardware drives.

✗ **(C):** Git preserves all changes safely until a human resolves the conflict; it never randomly deletes other functions.

✗ **(D):** Version control operates at the file tracking layer, completely separate from physical CPU instruction scheduling.

Question 85: Semantic Versioning & Software Updates

★★ IMPORTANT

Correct Answer: (A) — Increment the PATCH number (e.g., from `2.4.1` to `2.4.2`).

STEP-BY-STEP EXPLANATION

- In Semantic Versioning: • MAJOR version changes indicate incompatible, breaking API changes. • MINOR version changes add new functionality in a backwards-compatible manner. • PATCH version changes provide backwards-compatible bug fixes and security patches.
- Since this release strictly fixes defects without breaking or adding public interfaces, only the PATCH number increments (`2.4.1` -> `2.4.2`).

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** MAJOR version increments are reserved for breaking API changes that require consumers to rewrite their code.

✗ **(C):** Reverting to 0.0.0 falsely implies an unreleased pre-alpha build.

✗ **(D):** Omitting version numbers makes dependency management and tracking updates impossible.

Question 86: Trace Table: Algorithm with List Swaps

★★★★★ VERY IMPORTANT

Correct Answer: (A) — [10, 50, 20, 40]

STEP-BY-STEP EXPLANATION

- We trace each statement carefully: • Initial list: `[20, 50, 10, 40]` (index 1 is 20, index 3 is 10) • `temp <- items[1]`: `temp` saves the value 20. • `items[1] <- items[3]`: index 1 becomes 10. List is now `[10, 50, 10, 40]`. • `items[3] <- temp`: index 3 receives the saved value 20. List is now `[10, 50, 20, 40]`.
- This is the standard algorithm for swapping two elements in a list using a temporary variable.
- Elements at index 1 and index 3 have swapped positions.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** This is the original unchanged list before the swap occurred.

✗ **(C):** This would happen if the programmer forgot `items[3] <- temp` and left index 3 overwritten by index 1.

✗ **(D):** Index 2 (50) and index 4 (40) were never touched by this code.

Question 87: Accessibility Audits in Iterative Review

★★★ MANDATORY

Correct Answer: (A) — Screen readers synthesize the alt text into speech or braille for visually impaired students who cannot see the visual diagram.

STEP-BY-STEP EXPLANATION

- Screen reading software used by blind and low-vision individuals reads web content aloud sequentially.
- When encountering an image, the screen reader reads the `alt` text attribute.
- Providing clear, descriptive alternative text (e.g., 'Diagram of a plant cell with chloroplasts labeled in green') ensures that visually impaired students have equitable access to learning materials.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Alt text is a short text string; it does not change the compression or download speed of the image file.

✗ **(C):** Alt text is public HTML metadata used by search engines to index images; it does not encrypt anything.

✗ **(D):** Hardware monitors display pixel buffers directly; they do not read alt text attributes.

Question 88: Changelogs & User Communication

★★ IMPORTANT

Correct Answer: (A) — It informs users and administrators about new features, bug fixes, known issues, and potential breaking changes.

STEP-BY-STEP EXPLANATION

- A changelog provides transparent communication between developers and users.
- It lists what has changed (new features, resolved bugs, performance improvements, deprecated features) so users know what to expect and system administrators can prepare for updates safely.
- This transparency is a hallmark of professional software development.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Financial institutions handle payments; they do not review app changelog prose.
✗ **(C):** Previous releases remain archived in repository history and version control.
✗ **(D):** Digital changelogs have zero physical relationship with ambient atmospheric dust.

Question 89: Complex Trace Table: Accumulator with Conditional Filter

★★★★★ VERY IMPORTANT

Correct Answer: (A) — 10

STEP-BY-STEP EXPLANATION

- The algorithm iterates through each score in the list `scores <- [12, -4, 0, 8, -2, 10]`.
- The condition `IF (s > 0)` checks strictly for positive numbers.
- • For s = 12: 12 > 0 is TRUE -> positiveSum becomes 12, positiveCount becomes 1.
- • For s = -4: -4 > 0 is FALSE -> skipped.
- • For s = 0: 0 > 0 is FALSE (zero is not positive!) -> skipped.
- • For s = 8: 8 > 0 is TRUE -> positiveSum becomes 20, positiveCount becomes 2.
- • For s = -2: -2 > 0 is FALSE -> skipped.
- • For s = 10: 10 > 0 is TRUE -> positiveSum becomes 30, positiveCount becomes 3.
- After the loop ends, the average is calculated: `avgPositive <- 30 / 3 = 10`.
- `DISPLAY(avgPositive)` outputs 10.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** 4 is the total net sum divided by all 6 elements: (12 - 4 + 0 + 8 - 2 + 10) / 6 = 24 / 6 = 4. The code explicitly filters for positive numbers only (`s > 0`).
✗ **(C):** 5 would occur if 0 were incorrectly counted as positive (`positiveCount = 4`, 20 / 4 = 5).
✗ **(D):** 30 is the sum of positive numbers (`positiveSum`), not the average.

Question 90: Deprecation Warnings in Software Evolution

★★★ MANDATORY

Correct Answer: (A) — The procedure is still functional in the current version, but it is considered obsolete, will be removed in a future release, and developers are urged to migrate to a newer alternative.

STEP-BY-STEP EXPLANATION

- Software deprecation is a transparent warning mechanism.
- It informs developers that a feature or API is outdated and scheduled for removal in an upcoming major release.
- This gives developers advance notice to refactor their code and transition to the modern recommended API without breaking their live systems immediately.
- Managing technical debt through structured deprecation is standard industry practice.

WHY OTHER OPTIONS ARE INCORRECT

✗ **(B):** Deprecation is a routine engineering lifecycle state, not a criminal matter.
✗ **(C):** Deprecated software remains software code; it is not fabricated into silicon hardware.
✗ **(D):** Deprecated functions are often slower or less secure than modern alternatives, which is why they are being phased out.

Program Purpose, Function, and User Input Processing in a Healthcare App

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Purpose vs. Function:**
- The *purpose* of HydrateHero is to promote student-athlete health and physical well-being by helping users monitor their daily hydration and achieve water intake goals during sports seasons.
- The *function* describes the algorithmic behavior: reading text from input fields, validating positive numeric format, converting milliliters to ounces, adding to an accumulator variable `dailyTotalOz`, and updating UI labels and progress graphics in real time.

• **(b) Inputs & Outputs:**
- *Input:* `rawAmount` entered into `waterAmountInput` (string/number format, e.g., '500') and `unitType` selected from `unitDropdown` (text string: 'oz' or 'ml').
- *Output:* Text displayed on `dailyTotalLabel` (string showing current total vs. goal), visual update to the circular progress bar, or the auditory `playCelebrationSound()` when reaching the goal.

• **(c) Tracing '500 ml' Execution:**
- 1. User taps `logButton`, triggering the click event listener.
- 2. `rawAmount` ('500') and `unitType` ('ml') are extracted from the UI controls.
- 3. Condition `(isNumber(500) = true AND 500 > 0)` evaluates to TRUE.
- 4. `convertToOunces(500, "ml")` computes approximately 16.9 ounces.
- 5. `dailyTotalOz` increments by 16.9.
- 6. `updateProgressBar` redraws the fill percentage, and `dailyTotalLabel` is updated.
- 7. If `dailyTotalOz >= dailyGoalOz`, an auditory chime plays.

• **(d) Handling '-15':**
- Because `-15 > 0` evaluates to FALSE, the boolean expression evaluates to FALSE.
- Execution branches immediately to the `ELSE` block.
- `errorLabel` displays 'Please enter a valid positive number!'.
- *Validation Benefit:* Without this check, adding a negative number would decrease the athlete's logged water intake, creating a logic error that falsifies daily

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Clearly articulates the program purpose (assisting student-athletes to maintain healthy hydration) and distinguishes it from functional code mechanics.	**2 pts**
(b)	Correctly identifies an input (e.g., numeric string/number from waterAmountInput) and an output (e.g., visual label text, progress bar, audio chime) with accurate types.	**2 pts**
(c)	Traces execution through the positive branch: input extraction, unit conversion to ounces, accumulation in dailyTotalOz, UI update, and conditional celebration check.	**3 pts**
(d)	Identifies that negative input triggers the ELSE branch, setting the error label, and explains that this validation prevents corrupting dailyTotalOz with negative numbers.	**2 pts**

Iterative Development & User Feedback Integration in an Accessibility Tool

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Need for Iterative Development:**
- A non-iterative approach assumes all requirements are perfectly known upfront and builds the final software without intermediate validation.
- In this scenario, studio testing by a sighted developer failed to anticipate real-world usage barriers.
- Iterative development allows the developer to deploy early prototypes, observe authentic user friction, and refine the software across repeated cycles based on direct user feedback.

• **(b) Proposed Modifications:**
- 1. *Audio Guidance:* Implement computer vision edge-detection that emits auditory beeps or verbal prompts ('tilt camera down', 'page detected') to guide physical framing.
- 2. *Playback Controls:* Add prominent voice rate controls (0.75x, 1.0x, 1.25x) along with tap-anywhere gestures to pause and rewind speech.

• **(c) Next Iterative Cycle:**
- *Investigate:* Consult visually impaired classmates on preferred speech speeds and gesture habits.
- *Design:* Create a wireframe and gesture map for full-screen touch controls and audio feedback prompts.
- *Prototype:* Implement the gesture listener and speech rate parameters in an experimental software build.
- *Test:* Have the three classmates test reading textbook pages in varied ambient lighting, recording success rate and user satisfaction.

• **(d) Developer Blind Spots:**
- As a sighted person, the developer took visual page orientation and on-screen button locations for granted, assuming users could see to align the camera.
- Failing to consult diverse users early resulted in a system that was technically functional but practically inaccessible to its target audience.

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Explains that initial studio testing failed to uncover real-world usability challenges, and contrasts how iterative cycles allow responsive refinement based on empirical feedback.	**3 pts**
(b)	Identifies two valid accessibility improvements (e.g., orientation audio beeps, speed slider/pause controls, large haptic touch targets / screen-reader accessibility tags).	**2 pts**
(c)	Outlines the 4-phase cycle: investigating user needs, designing audio/haptic features, building an updated prototype, and conducting participatory testing with the classmates.	**2 pts**
(d)	Identifies the developer blind spot (a sighted developer assumes camera alignment is visually obvious), explaining how lack of diversity created product failure.	**2 pts**

Event-Driven Architecture & Concurrency in an Interactive School Voting Kiosk

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Event, Inputs & Outputs:**
- *Trigger Event:* A touch or mouse click event on `castVoteButton`.
- *Inputs:* `studentId` (alphanumeric string from text box) and `candidateSelected` (chosen candidate identifier).
- *Outputs:* Visual screen display (`displayConfirmationScreen` or `displayErrorScreen`), physical actuation output (`printPaperReceipt`), and auditory alert (`playSound`).

• **(b) Event-Driven vs. Sequential:**
- A sequential batch script runs strictly from top to bottom on a predetermined dataset and exits.
- An event-driven kiosk runs continuously in an event loop, remaining idle until a student interacts with the screen. When a touch event occurs, the registered handler executes, returns to idle, and awaits the next student.

• **(c) Concurrency & Event Queue:**
- When simultaneous votes arrive from Kiosk A and Kiosk B, the central server places them into an incoming FIFO (First-In, First-Out) *event queue*.
- Each event receives a high-resolution millisecond timestamp.
- The server processes queue items sequentially in atomic transactions: vote 1 is recorded and the student marked voted before vote 2 is processed.
- This prevents race conditions where duplicate ballots or corrupted tally totals could occur.

• **(d) Edge Case & Prevention:**
- *Edge Case:* A student leaves `idInput` completely blank ("") or enters letters instead of digits.
- *Prevention:* Add validation: `IF (length(studentId) = 6 AND isDigitsOnly(studentId))`. If invalid, immediately display 'Invalid Student ID format' before making server database calls.

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Accurately identifies the trigger event (mouse/touch click on castVoteButton), inputs (studentId, candidateSelected), and outputs (confirmation/error screen, paper receipt, audio alert).	**2 pts**
(b)	Explains that sequential scripts run once from start to finish, whereas event-driven models idle asynchronously until user interactions trigger handlers.	**2 pts**
(c)	Explains the event queue/event loop mechanism: incoming events are timestamped and queued sequentially, preventing simultaneous write collisions and race conditions.	**3 pts**
(d)	Identifies a valid edge case (e.g., blank ID string, non-numeric ID, SQL injection characters) and provides a concrete validation check (e.g., length check, regex pattern matching).	**2 pts**

Collaborative Pair Programming, Role Alternation & Bias Elimination in Computer Vision

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Pair Programming Violations & Fixes:**
- *Violation 1 (Monopolized Typing):* Developer X typed continuously for 2 hours without switching roles. *Fix:* Set a 15–20 minute timer to switch driver and navigator roles regularly.
- *Violation 2 (Navigator Disengagement):* Developer Y checked personal email rather than actively reviewing code. *Fix:* The navigator must actively watch the screen, speak aloud, verify logic, check for edge cases, and maintain continuous dialogue.

• **(b) Dataset Bias & Cafeteria Failure:**
- *Bias:* The training data represents an artificial, idealized scenario (clean cans, white background, fluorescent light).
- *Performance:* In a real school cafeteria, recycling items are crumpled, crushed, partially filled with liquids, covered in food residue, and viewed under varying ambient lighting and angles. The model will frequently misclassify or fail to recognize real waste items.

• **(c) Community Collaboration:**
- The team should collaborate with students, teachers, and custodial staff to photograph hundreds of real waste items directly from cafeteria bins across multiple days.
- Including crushed cans, grease-stained paper, different brands, and diverse angles creates a robust, representative training dataset.

• **(d) Strategic Role of the Navigator:**
- While the driver focuses on writing tensor syntax and neural network layers, the navigator's job is strategic oversight.
- The navigator should have asked: 'Wait, why are we only photographing clean cans on white desks? What happens when a student tosses a crushed soda can into a dark blue bin?'

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Identifies lack of role switching (driver monopolized keyboard for 2 hours) and navigator disengagement; specifies structured timer rotations (every 15–20 mins) and active verbal review.	**3 pts**
(b)	Explains dataset bias (only clean cans on white backgrounds under studio light) and predicts high cafeteria failure rate (crushed items, soiled labels, varying shadows, dirty recycling bins).	**2 pts**
(c)	Describes gathering images of diverse, crushed, half-eaten, and real-world soiled recyclables from cafeteria bins and consulting cafeteria staff/custodians.	**2 pts**
(d)	Explains that the navigator should step back from immediate coding, evaluate the broader problem domain, and question whether the dataset matches real-world cafeteria operating conditions.	**2 pts**

Comprehensive Error Classification & Debugging in an E-Commerce Cart Engine

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Error Classifications:**
- *Defect 1 (Logic Error):* The program compiles cleanly and executes without crashing, but computes the wrong mathematical tax balance due to a flawed financial formula.
- *Defect 2 (Syntax Error):* The code violates the grammatical syntax rules of the programming language (missing `)`). The compiler halts immediately and refuses to run the code.
- *Defect 3 (Runtime Error):* The code is syntactically valid and compiles, but while running, attempts integer division by zero, causing an unhandled hardware/runtime exception that aborts execution.

• **(b) Why Logic Errors Are Dangerous:**
- Syntax errors are caught automatically by compilers, and runtime errors announce themselves loudly with crash stack traces.
- Logic errors are silent: the software appears to run normally, but outputs erroneous data.
- In commercial e-commerce or financial systems, silent logic errors can overcharge thousands of customers or cause regulatory non-compliance before being discovered.

• **(c) Corrected Code for Defect 1:**
``` discountedTotal <- subTotal - discount totalCost <- discountedTotal + (discountedTotal * taxRate) ```

• **(d) Defensive Guard for Defect 3:**
``` IF (inventoryCount > 0) { ratio <- totalOrders / inventoryCount } ELSE { ratio <- 0 DISPLAY("Notice: Item is currently out of stock.") } ```

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Correctly classifies Defect 1 as a logic error, Defect 2 as a syntax error, and Defect 3 as a runtime error with clear, accurate justifications.	**3 pts**
(b)	Explains that logic errors do not generate compiler warnings or runtime crashes, allowing incorrect computations to go unnoticed into production.	**2 pts**
(c)	Provides correct formula applying tax to discounted subtotal: discountedSubtotal <- subTotal - discount; totalCost <- discountedSubtotal + (discountedSubtotal * taxRate).	**2 pts**
(d)	Writes valid defensive conditional: IF (inventoryCount > 0) or IF (inventoryCount != 0) before performing division, handling the zero case safely.	**2 pts**

Systematic Testing, Boundary Conditions & Test Suite Coverage in a Ride-Share Fare Calculator

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Purpose & Return Type:**
- *Purpose:* To compute the monetary cost of a ride based on distance traveled with tiered mileage rates and peak rush-hour demand multipliers.
- *Return Type:* A real/decimal number representing dollars, or `-1` indicating an invalid distance error.

• **(b) Four-Case Test Suite:**
- *Case 1 (Invalid):* `calculateFare(-2, false)` -> Expected: `-1` (triggers `distanceMiles <= 0`).
- *Case 2 (Short Normal):* `calculateFare(3, false)` -> `base 3.50 + (3 * 2.00 = 6.00) = 9.50` -> Expected: `9.50`.
- *Case 3 (Long Normal):* `calculateFare(8, false)` -> `base 3.50 + 10.00 + (3 * 1.50 = 4.50) = 18.00` -> Expected: `18.00`.
- *Case 4 (Rush Hour):* `calculateFare(2, true)` -> `(3.50 + 4.00) * 1.25 = 7.50 * 1.25 = 9.375` -> Expected: `9.375` (or $9.38).

• **(c) Boundary Analysis:**
- The boundaries are `distanceMiles = 0` (valid vs. invalid) and `distanceMiles = 5` (single rate vs. tiered rate).
- At `distanceMiles = 5`: `mileageFee = 5 * 2.00 = 10.00`.
- At `distanceMiles = 5.1`: enters the `ELSE` branch: `10.00 + (0.1 * 1.50 = 0.15) = 10.15`.
- Testing on either side of 5 confirms that the inequality `<= 5` transitions smoothly without pricing cliff anomalies.

• **(d) Trace for `calculateFare(10, true)`:**
- 1. `baseFare = 3.50`
- 2. `distanceMiles <= 0` (10 <= 0) is FALSE.
- 3. `distanceMiles <= 5` (10 <= 5) is FALSE -> enters `ELSE` branch.
- 4. `mileageFee = (5 * 2.00) + ((10 - 5) * 1.50) = 10.00 + (5 * 1.50) = 10.00 + 7.50 = 17.50`.
- 5. `totalFare = 3.50 + 17.50 = 21.00`.
- 6. `isRushHour = true` is TRUE -> `totalFare = 21.00 * 1.25 = 26.25`.
- 7. Return value: **$26.25**.

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Accurately defines purpose (computing total ride fare based on distance and peak demand) and identifies return type as a numeric/decimal value (or error code -1).	**2 pts**
(b)	Constructs four valid test cases covering all structural branches with correct input parameters and verified expected outputs.	**3 pts**
(c)	Identifies boundary values (0 and 5) and explains that 5 vs. 5.1 tests the boundary where the tiered pricing rate transitions from $2.00/mi to $1.50/mi.	**2 pts**
(d)	Correctly evaluates calculateFare(10, true): base $3.50 + $10.00 + $7.50 = $21.00; rush hour 1.25x = $26.25.	**2 pts**

Procedural Abstraction & Modular Decomposition in an Environmental Sensor Array

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Procedural Abstraction:**
- Procedural abstraction is the practice of encapsulating a sequence of instructions into a named procedure that can be invoked repeatedly without the caller needing to know its internal code.
- The main program simply calls `calibrateReadings(rawVoltages)` without managing raw analog-to-digital formulas.

• **(b) Collaborative Advantages:**
- *Parallel Work:* One student can work on cellular telemetry protocols in `transmitTelemetry()` while another refines the chemical pollution thresholds in `detectPollutionThresholds()`.
- *Clear Interface Contracts:* As long as inputs and return data formats agree, students work independently without merge conflicts.

• **(c) Impact of Hardware Sensor Changes:**
- Only `calibrateReadings(rawVoltages)` must be modified.
- Because the procedure encapsulates the voltage-to-pH conversion and returns the same standardized calibrated data object, `detectPollutionThresholds()` and `transmitTelemetry()` continue functioning without changing a single line of code.

• **(d) Unit Testing Benefits:**
- Instead of deploying the physical buoy into deep lake water to test everything at once, students can pass simulated voltage values to `calibrateReadings()` on a workbench to verify accuracy.
- This isolates bugs before real-world deployment.

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Defines procedural abstraction (hiding low-level implementation details behind a callable interface) and maps it to the refactored procedures.	**2 pts**
(b)	Explains collaboration advantages: independent parallel development, clear input/output contracts, reduced merge conflicts, and isolated code responsibilities.	**3 pts**
(c)	Identifies that only calibrateReadings() requires updating; explains that modular encapsulation shields the reading, detection, and transmission procedures from internal calibration formula changes.	**2 pts**
(d)	Explains that each procedure can be tested in isolation with mock data (unit testing) without needing the physical buoy in the water.	**2 pts**

Trace Table Analysis, State Verification & Defect Isolation in a Grade Filtering Algorithm

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Logic Error in Test 3:**
- In Test 3, all students were absent (`-1`). Returning `0` is a logic error because nobody took the exam.
- A score of `0` falsely indicates that a student took the test and failed with zero credit, distorting grade statistics.

• **(b) Trace Table for Test 2 (`[-1, 84, -1, 91, 78]`):**
- *Initial state:* `maxScore = 0`
- *Iteration 1 (`s = -1`):* `(-1 > 0)` is FALSE -> `maxScore = 0`
- *Iteration 2 (`s = 84`):* `(84 > 0)` is TRUE -> `maxScore = 84`
- *Iteration 3 (`s = -1`):* `(-1 > 84)` is FALSE -> `maxScore = 84`
- *Iteration 4 (`s = 91`):* `(91 > 84)` is TRUE -> `maxScore = 91`
- *Iteration 5 (`s = 78`):* `(78 > 91)` is FALSE -> `maxScore = 91`
- *Final Return:* `91`.

• **(c) Corrected Algorithm:**
``` PROCEDURE findHighestScore(scoresList) { maxScore <- -1 foundValidScore <- false FOR EACH s IN scoresList { IF (s != -1) { IF (foundValidScore = false OR s > maxScore) { maxScore <- s foundValidScore <- true } } } IF (foundValidScore = false) { RETURN "No scores present" } RETURN maxScore } ```

• **(d) Defect in Sub-Zero Context:**
- Initializing `maxScore <- 0` assumes all valid values are non-negative.
- In a list like `[-15, -8, -22]`, every value is strictly less than 0.
- The condition `(s > maxScore)` will never evaluate to true, and the procedure will erroneously return `0`, even though 0 never appeared in the dataset and the true maximum was `-8`.

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Explains that 0 implies a student took the test and scored zero points, which falsifies records when in reality all students were absent.	**2 pts**
(b)	Accurately traces Test 2 across all 5 iterations with correct boolean checks and variable updates.	**3 pts**
(c)	Rewrites algorithm with a flag or counter to verify if at least one valid non-(-1) score was processed, returning 'No scores present' when none exist.	**2 pts**
(d)	Explains that if all temperatures are negative, maxScore remains 0, erroneously reporting a warm 0 degrees when the true maximum was -8.	**2 pts**
```

Intellectual Property, Open Source Licensing & Code Attribution in a Robotics Controller

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Ethical & Legal Violations:**
- *License Infringement:* The MIT license requires preserving the original copyright notice and permission text. Stripping this header violates the legal license agreement.
- *Academic Plagiarism / Fraud:* Claiming in his project report that he wrote the algorithm from scratch when he copied it is academic dishonesty and false representation.

• **(b) Proper Integration & Attribution:**
- 1. Retain the original copyright notice and MIT license text at the top of the imported library file.
- 2. In his project documentation and report, clearly state: 'Motor timing is powered by the open-source AccurateStepper library by [Author Name], licensed under MIT [URL].'
- 3. Distinctly document which parts Carlos personally wrote (e.g., obstacle detection and path planning) versus third-party libraries.

• **(c) MIT vs. GPL (Copyleft):**
- *MIT License:* Highly permissive; allows anyone to use, modify, and even incorporate the code into proprietary, closed-source commercial software without releasing their own source code.
- *GNU GPL (Copyleft):* Reciprocal license; mandates that any derivative work that incorporates GPL code must also be licensed under the GPL and have its full source code made publicly available.

• **(d) AP CSP Create Task Rules:**
- Carlos is allowed to *use* the MIT stepper library to make his rover move.
- However, for the written response prompts requiring explanation of an algorithm with sequencing, selection, and iteration, Carlos *cannot* submit or describe `AccurateStepper`.
- He must select and explain an algorithm that he *independently developed* (such as his obstacle avoidance algorithm).

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Identifies copyright infringement (violating MIT license terms by stripping copyright notices) and academic plagiarism (falsely claiming original authorship in his report).	**2 pts**
(b)	Details proper attribution: retaining original copyright headers, citing library name, author, URL, and license in project documentation, and clarifying his original contributions.	**3 pts**
(c)	Contrasts permissive licenses (MIT allows proprietary closed derivative works) with copyleft (GPL requires derivative works to remain open under the same license).	**2 pts**
(d)	Explains that under AP CSP Create Task rules, Carlos must explain and submit an algorithm that he personally wrote, not imported library code.	**2 pts**

End-to-End Creative Development Lifecycle: From Investigation to Maintenance in a Disaster Relief Portal

9 Points Total

OFFICIAL AP MODEL RESPONSE:

• **(a) Phase 1: Investigation & Reflection:**
- Interview disaster relief workers, local shelter organizers, and storm victims to identify urgent priorities (e.g., immediate need for baby formula, insulin refrigeration, pet-friendly shelter).
- Uncover critical real-world operational constraints, such as intermittent cellular connectivity, dead phone batteries, and high stress levels among users.

• **(b) Phase 2: Design with Wireframes & Flowcharts:**
- Paper wireframes and flowcharts allow developers, emergency workers, and volunteers to physically walk through the shelter-matching workflow in minutes.
- This validates interface clarity and decision logic before coding, preventing teams from wasting critical emergency hours building confusing screens.

• **(c) Phase 3: Incremental Deployment (MVP):**
- *Hour 0–24 (Increment 1):* Deploy a lightweight, fast Minimum Viable Product (MVP) containing just two basic features: 'I Need Shelter' and 'I Have Shelter' with direct SMS/phone connections.
- *Days 2–4 (Increment 2):* Add search filters for medical supplies, food allergies, and wheelchair accessibility.
- *Days 5–7 (Increment 3):* Integrate GPS map clustering, real-time supply inventory, and multi-language translations.

• **(d) Phase 4: Testing & Reliability:**
- 1. *Load/Stress Testing:* Simulate thousands of simultaneous requests to verify server caching and ensure the database does not crash under surge traffic.
- 2. *Degraded Network / Offline Testing:* Throttle connections to 2G speeds and test offline caching (Service Workers) to ensure disaster victims with poor cellular reception can still view emergency shelter locations.

OFFICIAL AP SCORING RUBRIC (9 POINTS BREAKDOWN):

Part	Scoring Criteria & Benchmark Expectation	Points
(a)	Describes consulting displaced families, disaster volunteers, and emergency first responders to uncover real-world constraints (power outages, low bandwidth, emergency medical priority).	**2 pts**
(b)	Explains that wireframes and flowcharts establish visual consensus, test user navigation, and clarify system interfaces without wasting precious emergency coding hours.	**2 pts**
(c)	Explains deploying an initial increment (basic shelter registration and phone directory) immediately, followed by iterative additions (mapping, supply filters, multi-lingual support).	**3 pts**
(d)	Identifies load/stress testing (simulating thousands of concurrent emergency requests) and offline/low-bandwidth testing (testing 2G/3G connectivity and cellular drops).	**2 pts**

Essential Unit 1 Creative Development Reference Framework

Core AP exam pseudocode conventions, defect classifications, development methodologies, and licensing taxonomies.

- **AP CSP Pseudocode Conventions:** `a <- expression` evaluates expression and assigns to variable a. `DISPLAY(expression)` displays value followed by a space. `INPUT()` accepts user input. 1-based indexing is standard for lists on the AP exam: `aList[1]` accesses the first element.
- **Defect Typologies:**
  - *Syntax Error:* Violates grammatical structure of the language; detected before execution; prevents running.
  - *Runtime Error:* Occurs during execution, causing the program to abnormally crash or terminate (e.g., division by zero, null pointer, index out of bounds).
  - *Logic Error:* Program compiles and runs without crashing, but computes or displays the wrong result due to flawed algorithm design.
  - *Overflow Error:* Occurs when a calculated integer exceeds the maximum finite bit representation limit.
  - *Round-off Error:* Occurs when fractional real numbers cannot be represented with exact precision in binary floating point.
- **Program Purpose vs. Function:**
  - *Purpose:* The problem being solved or the value provided to users (e.g., helping athletes maintain hydration).
  - *Function:* How the code operates algorithmically (e.g., reading numeric text inputs, validating formats, converting units, updating UI controls).
- **Development Methodologies:**
  - *Iterative Development:* Repeating cycles of investigation, design, coding, testing, and feedback.
  - *Incremental Development:* Building the system piece by piece in small, manageable, fully functional modules.
  - *Pair Programming:* Two programmers at one station; the *driver* writes code while the *navigator* reviews strategy, catches bugs, and plans architecture.
- **Intellectual Property & Licensing:**
  - *Copyright:* Exclusive legal right protecting original works of authorship upon creation.
  - *Creative Commons (CC):* Public licenses enabling creators to grant standard copyright permissions.
  - *CC BY:* Requires attribution. *CC BY-SA:* Requires attribution and sharing alike. *CC BY-NC:* Commercial use forbidden.
  - *Open Source (MIT / GPL):* Free distribution of source code with permissive or copyleft requirements.
  - *Public Domain:* Works with expired or forfeited copyright; usable freely without permissions.

Disclaimer: This educational publication contains original AP Computer Science Principles-aligned practice questions developed strictly for pedagogical preparation and exam mastery. It is not affiliated with, endorsed by, or approved by the College Board. "AP" and "Advanced Placement" are registered trademarks of the College Board.



Thank You, Future Computer Scientists!

DEDICATED TO ALL STUDENTS STRIVING FOR AP EXCELLENCE

You have successfully completed all 100 master questions of the **AP Computer Science Principles Unit 1 Master Set**. By mastering collaborative pair programming, algorithmic purpose vs. function, event-driven architecture, iterative refinement, robust debugging, and ethical intellectual property practices, you have built the foundational bedrock of modern computing.

> "Code is not merely instructions for silicon chips — it is a canvas for human creativity, collaboration, and solving real-world problems for a better society."

BEST WISHES FOR A SCORE OF 5 ON THE AP COMPUTER SCIENCE PRINCIPLES EXAM!